Tự ôn tập phỏng vấn về lập trình (Coding Interview University)

Bản gốc:

- English

Tác giả gốc: John Washam

Đóng góp cho bản dịch tiếng Việt:

- Lê Tiến Tài - @letientai299
- Võ Tường Thọ - @thovo
- Lê Tấn Đăng Khoa - @dksdc
- Trương Đức Duy - @dauruy
- Lương Đăng Hải - @jarvisluong
- Hiền Vương - @duchienvuong
- Nguyễn Đức Đông

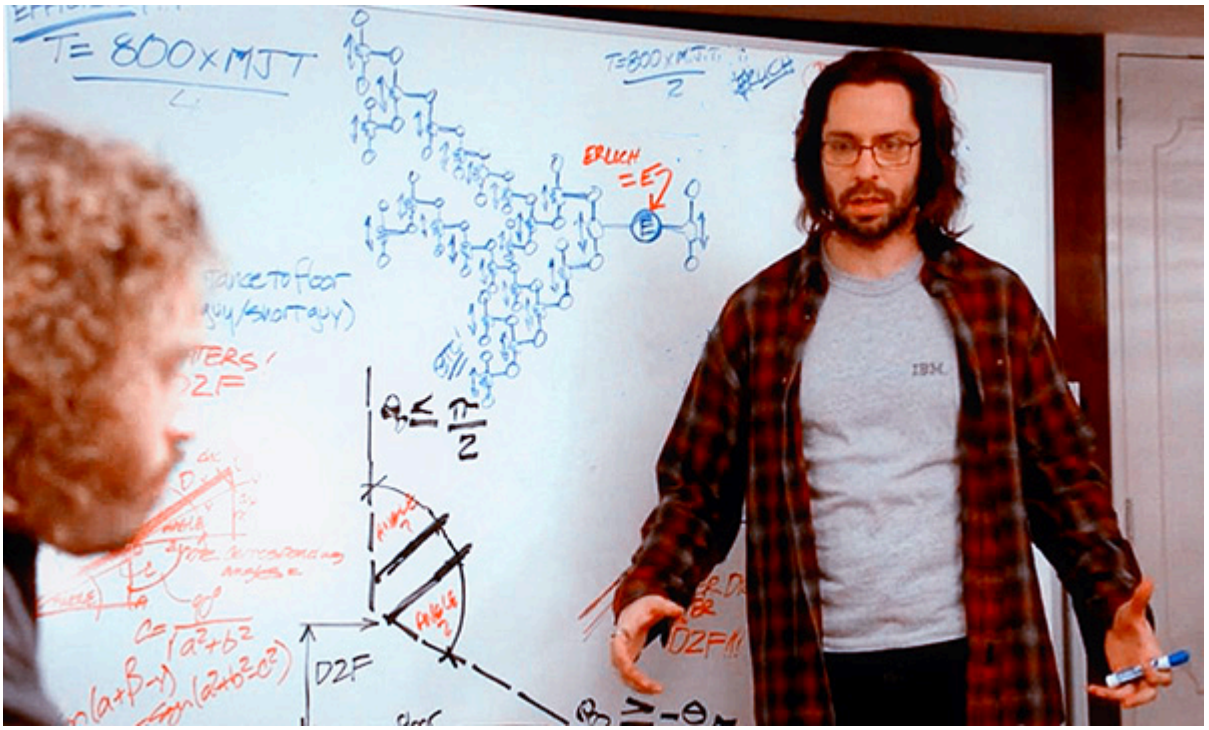
Ghi chú riêng cho việc duy trì và cập nhật bản dịch tiếng Việt:

- Bản dịch này nhằm mục đích khuyến khích các bạn trẻ yêu thích công nghệ nhưng chưa vững tiếng Anh dễ tiếp cận, và tìm được hướng nghiên cứu. Để đi xa hơn trong ngành công nghệ thông tin (CNTT), sớm hay muộn, bạn cũng cần phải trau dồi vốn tiếng Anh của mình. Vì vậy, các thuật ngữ chuyên ngành, mình xin được giữ nguyên gốc. Ví dụ như: `stack`, `heap`, `queue`,...
- Mình cố gắng dịch thoát nghĩa, sao cho các bạn với ít kiến thức công nghệ thông tin nhất cũng có thể hiểu được. Trong quá trình dịch khó có thể trách khỏi sai sót, xin được lượng thứ.
- Mọi ý kiến, đóng góp về bản dịch, vui lòng tạo một issue mới hoặc bạn có thể chỉnh sửa và tạo Pull Request, đồng thời cc trực tiếp các dịch giả để kiểm tra.

> Ban đầu, đây chỉ là một danh sách to-do (danh sách các việc cần làm) ngắn về các chủ đề phải ôn tập của tôi, để trở thành một kỹ sư phần mềm. Nhưng rồi nó lớn dần nên như ngày nay. Sau khi đi hết con đường này, tôi đã được tuyển vào vị trí Software Development Engineer ở Amazon! Bạn có lẽ không cần phải học nhiều như tôi đã học. Nhưng dù sao, mọi thứ bạn cần ở đây.
>
> I studied about 8-12 hours a day, for several months. This is my story: Why I studied full-time for 8 months for a Google interview
>
> **Lưu ý:** Bạn sẽ không cần phải học nhiều như tôi đã làm. Tôi đã lãng phí rất nhiều thời gian vào những thứ mà tôi không cần biết. Thêm thông tin về điều này ở phía dưới. Tôi sẽ giúp bạn đạt được mục tiêu mà không lãng phí thời gian quý giá của bạn.
>
> Những chủ đề này sẽ chuẩn bị cho bạn nền tảng kiến thức vững vàng cho cuộc phỏng vấn kỹ thuật tại bất kỳ công ty phần mềm nào, bao gồm cả những gã khổng lồ như: Amazon, Facebook, Google hay Microsoft.

> *Chúc may mắn!*

Giới thiệu



Đây là kế hoạch học tập kéo dài nhiều tháng của tôi để trở thành một software engineer cho một công ty lớn.

Yêu cầu:

- Một chút kinh nghiệm về mã hóa (biến, vòng lặp, phương thức/hàm, v.v.)
- Tính kiên nhẫn
- Thời gian

Lưu ý rằng đây là một kế hoạch dành cho **software engineering**, không phải frontend engineering hay full-stack. Thực sự có một siêu lộ trình và khóa học cho những career path đó ngoài kia (xem https://roadmap.sh/ để biết thêm chi tiết).

Có rất nhiều điều để học trong một chương trình Khoa học Máy tính tại trường đại học, nhưng chỉ cần biết khoảng 75% là đủ tốt cho một cuộc phỏng vấn, vì vậy đó là những gì tôi đề cập ở đây. Để có một chương trình tự học Khoa học Máy tính hoàn chỉnh, các tài nguyên cho kế hoạch học tập của tôi đã được đưa vào Lộ trình Khoa học Máy tính của Kamran Ahmed: https://roadmap.sh/computer-science

---

Mục lục

Kế hoạch học tập

- Giới thiệu?
- Vì sao tôi cần tài liệu này?
- Sử dụng tài liệu này như thế nào?
- Đừng nghĩ rằng bạn không đủ thông minh
- Về nguồn video

- Chọn ngôn ngữ lập trình cho cuộc phỏng vấn
- Sách về Cấu Trúc Dữ Liệu và Thuật Toán
- Sách Chuẩn Bị Phỏng Vấn
- Đừng phạm sai lầm của tôi
- Những phần không được đề cập
- Kế hoạch hằng ngày
- Luyện tập các câu hỏi coding
- Giải bài toán lập trình

Nội dung học tập

- Độ phức tạp của thuật toán / Big-O / Phân tích tiệm cận
- Cấu trúc dữ liệu
  - Arrays
  - Linked Lists
  - Stack
  - Queue
  - Hash table
- Kiến thức bổ sung
  - Tìm kiếm nhị phân
  - Toán tử trên bit
- Cây
  - Cây - Ghi chú và kiến thức nền
  - Cây tìm kiếm nhị phân
  - Heap / Priority Queue / Binary Heap
  - Cây tìm kiếm cân bằng (một chủ đề chung, không đi sâu vào chi tiết)
  - Duyệt cây: preorder, inorder, postorder, BFS, DFS
- Sắp xếp
  - Sắp xếp chọn (Selection Sort)
  - Sắp xếp chèn (Insertion Sort)
  - Sắp xếp chọn vun đống (Heapsort)
  - Sắp xếp nhanh (Quicksort)
  - Sắp xếp trộn (Merge Sort)
- Đồ thị
  - có hướng
  - vô hướng
  - ma trận kề
  - danh sách kề
  - duyệt đồ thị: BFS, DFS
- Kiến thức bổ sung
  - Đệ quy
  - Quy hoạch động
  - Mẫu thiết kế
  - Tổ hợp và Xác Suất
  - NP, NP-Complete và thuật toán xấp xỉ gần đúng
  - Cách máy tính xử lý một chương trình
  - Bộ nhớ cache

    - Process và Thread
    - Kiểm thử phần mềm
    - Tìm kiếm và xử lý chuỗi
    - Tries
    - Cách biểu diễn số thực
    - Unicode
    - Endianness
    - Mạng máy tính
- Tổng kết

Tìm việc

- Lý lịch (Resume) của bạn
- Khi bạn tiến gần đến kỳ phỏng vấn
- Quy trình phỏng vấn & các bước chuẩn bị tổng quát
- Hãy nghĩ đến những thứ bạn sẽ được hỏi
- Chuẩn bị câu hỏi dành cho phỏng vấn viên
- Khi bạn được nhận việc

---------- Những mục dưới đây là tuỳ chọn ----------

Optional Extra Topics & Resources

- Sách bổ sung
- Thiết kế hệ thống, Khả năng mở rộng, Xử lý dữ liệu (Nếu bạn có hơn 4 năm kinh nghiệm)
- Học thêm
    - Trình biên dịch
    - Emacs và vi(m)
    - Các công cụ chạy trên dòng lệnh của Unix
    - Lý thuyết thông tin
    - Parity & Hamming Code
    - Entropy
    - Mã hóa
    - Thuật toán nén
    - Bảo mật
    - Trình dọn rác
    - Lập trình song song
    - Messaging, Serialization, and Queueing Systems
    - A*
    - Fast Fourier Transform
    - Bloom Filter
    - HyperLogLog
    - Locality-Sensitive Hashing
    - van Emde Boas Trees
    - Augmented Data Structures
    - N-ary (K-ary, M-ary) trees
    - Balanced search trees
        - AVL trees

        - Splay trees
        - Red/black trees
        - 2-3 search trees
        - 2-3-4 Trees (aka 2-4 trees)
        - N-ary (K-ary, M-ary) trees
        - B-Trees
    - k-D Trees
    - Skip lists
    - Network Flows
    - Disjoint Sets & Union Find
    - Math for Fast Processing
    - Treap
    - Linear Programming
    - Geometry, Convex hull
    - Discrete math
    - Machine Learning
    - Go
- Đọc thêm về một số đề tài
- Các chuỗi Video
- Các khóa học khoa học máy tính

Vì sao tôi cần tài liệu này?

Nếu bạn muốn làm kỹ sư phần mềm cho một công ty lớn, đây là những điều bạn phải biết.

Nếu bạn đã bỏ lỡ cơ hội để có được bằng cử nhân khoa học máy tính, giống như tôi, điều này sẽ giúp bạn bắt kịp và tiết kiệm bốn năm cuộc đời của bạn.

Khi tôi bắt đầu dự án này, tôi không phân biệt được stack và heap, không biết về Big-O, không có khái niệm gì về cây (`tree`) hay việc duyệt đồ thị (`graph traversal`). Và nếu buộc phải viết code cho một thuật toán sắp xếp, tôi đảm bảo rằng nó sẽ không chạy tốt.

Tất cả các cấu trúc dữ liệu tôi từng sử dụng đều được cài đặt sẵn trong ngôn ngữ lập trình và tôi không nhất thiết phải biết chúng làm việc như thế nào. Tôi chưa từng phải tự quản lý vùng nhớ, trừ khi một Process đang chạy ném lỗi "hết bộ nhớ" (`out of memory`), và sau đó tôi phải tìm một cách giải quyết khác. Tôi từng sử dụng mảng nhiều chiều vài lần trong đời, và hàng ngàn mảng kết hợp (`associate arrays`). Nhưng thực sự tôi chưa từng tự mình xây dựng một cấu trúc dữ liệu nào.

Đây là một kế hoạch dài hơi, sẽ tốn của bạn vài tháng. Nếu bạn đã quen với nhiều nội dung trong này, bạn sẽ mất ít thời gian hơn.

Sử dụng tài liệu này như thế nào?

Tất cả những gì dưới đây là một bản phác thảo, và bạn nên tiếp cận các mục theo thứ tự từ trên xuống dưới.

> Phần này được viết lại khá nhiều để thuận tiện cho các bạn tiếp cận. Dựa theo bản gốc, tác giả có vẻ như cũng đang cố hướng dẫn cho người mới dùng git.

Bạn có thể bỏ qua mục này nếu đã có kiến thức về Git, Github và Github Flavored Markdown

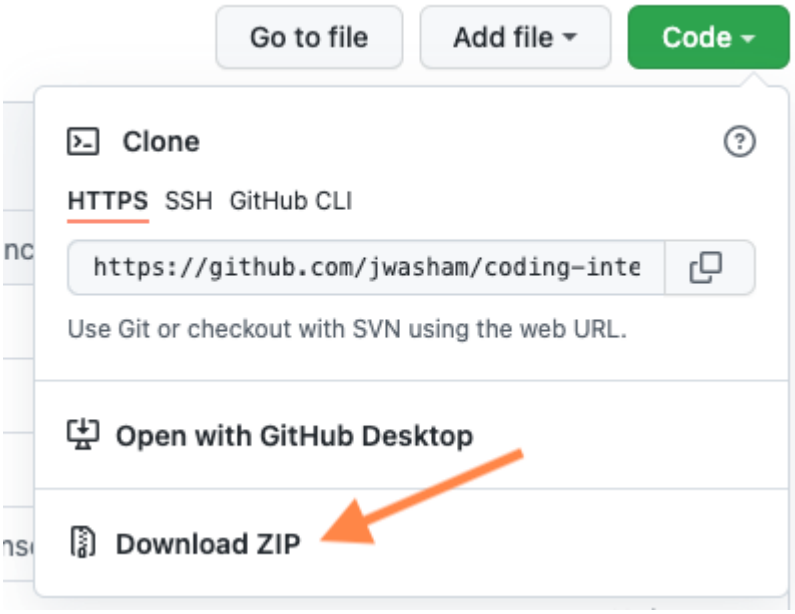
Nếu bạn chưa biết về git thì vui lòng tham khảo các bài hướng dẫn sau để nắm cách sử dụng:

- Tiếng Anh: git - the simple guide
- Tiếng Việt: Sổ tay git cho người mới bắt đầu (Việt hóa từ nội dung với link trên)

Nếu bạn không muốn sử dụng git

Trên trang này, nhấp vào nút Code gần phía trên, sau đó nhấp vào "Download ZIP". Giải nén file và bạn có thể làm việc với các file văn bản.

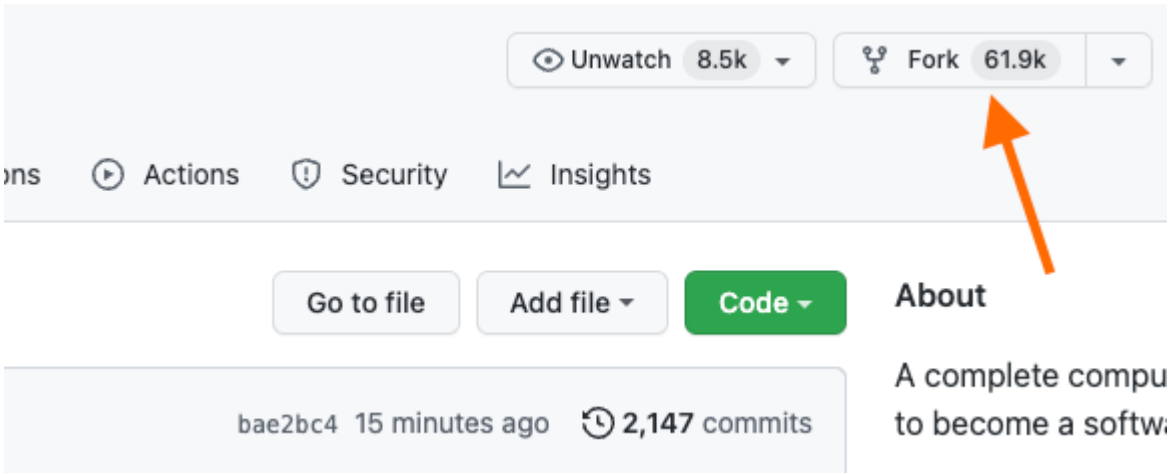
Nếu bạn mở trong một trình soạn thảo code hiểu markdown, bạn sẽ thấy mọi thứ được định dạng đẹp mắt.



Nếu bạn muốn làm quen với git

Tiếp theo, bạn cần biết cách gắp (fork) một repo trên github:

1. ***Fork GitHub repo:*** `https://github.com/jwasham/coding-interview-university` bằng cách clicking vào nút Fork.



- Tiếng Việt: Cách gắp (fork) một repo trên github (Việt hoá từ nội dung với link trên)

1. Clone bản fork của bạn về máy tính cá nhân.

```
git clone https://github.com/<YOUR_GITHUB_USERNAME>/coding-interview-university.git
cd coding-interview-university
git remote add upstream https://github.com/jwasham/coding-interview-university.git
git remote set-url --push upstream DISABLE  # để bạn không đẩy Process cá nhân của mình trở lại repo gốc
```

- Chạy các dòng lệnh sau
- Tạo một branch mới để đánh dấu tiến độ của bạn:

```
git checkout -b progress
```

1. Check các phần đã hoàn thành bằng cách thêm `x` vào giữa cặp ngoặc vuông (`[ ]`), như thế này: `[x]`.

- Chạy `git add .` để bắt đầu lưu lại các thay đổi.
- Chạy `git commit -m "commit message"` . Thay `commit message` với ghi chú của bạn cho sự thay đổi đó.
- Đồng bộ thay đổi với bản fork trên Github của bạn bằng `git push origin main`.

Đừng nghĩ rằng bạn không đủ thông minh

- Các kỹ sư phần mềm thành công thì thông minh, nhưng nhiều người lại tự ti rằng họ không đủ thông minh.
- Những video sau đây có thể giúp bạn vượt qua nỗi tự ti này:
  - Bí mật của của Thiên Tài Lập Trình (The myth of the Genius Programmer) - video
  - ulie Pagano: Đi một mình rất nguy hiểm - Cuộc chiến với con quái vật vô hình trong công nghệ
  - Hãy tin bạn có thể thay đổi

Về nguồn video

Một vài video chỉ xem được khi bạn tham gia vào các lớp học online trên Coursera, EdX, hay Lynda.com. Các lớp đó được gọi là MOOC. Đôi khi các lớp chưa mở, và bạn phải đợi một vài tháng đến khi chúng được mở lại, do đó bạn không thể truy cập vào video được. Lynda.com thì không miễn phí.

Tôi sẽ rất cảm kích sự hỗ trợ của các bạn trong việc thêm các nguồn video miễn phí và luôn sẵn có, ví dụ như Youtube, để hỗ trợ nguồn video từ các khóa học online. Tôi cũng rất thích xem các bài giảng của các trường đại học.

Chọn ngôn ngữ lập trình cho cuộc phỏng vấn

Bạn sẽ cần phải chọn một ngôn ngữ lập trình cho các cuộc phỏng vấn lập trình mà bạn tham gia, nhưng bạn cũng cần tìm một ngôn ngữ mà bạn có thể sử dụng để nghiên cứu các khái niệm khoa học máy tính.

Ưu tiên là ngôn ngữ đó sẽ giống nhau, để bạn chỉ cần thành thạo một ngôn ngữ.

Đối với Kế Hoạch Học Tập này

Khi tôi thực hiện kế hoạch học tập, tôi đã sử dụng 2 ngôn ngữ cho hầu hết phần này: C và Python

- C: Ngôn ngữ bậc thấp. Cho phép bạn làm việc với con trỏ và cấp phát/giải phóng bộ nhớ, vì vậy bạn cảm nhận được cấu trúc dữ liệu và thuật toán trong tâm can của mình. Trong các ngôn ngữ cấp cao hơn như Python hoặc Java, những điều này được ẩn đi. Trong công việc hàng ngày, điều đó thật khủng khiếp, nhưng khi bạn đang học cách xây dựng những cấu trúc dữ liệu cấp thấp này, thật tuyệt vời khi cảm thấy gần với phần cứng.
    - C có mặt ở khắp mọi nơi. Bạn sẽ thấy các ví dụ trong sách, bài giảng, video, *ở khắp mọi nơi* trong khi bạn đang học.
    - Ngôn ngữ lập trình C, Ấn bản thứ 2
        - Đây là một cuốn sách ngắn, nhưng nó sẽ giúp bạn hiểu rõ về ngôn ngữ C và nếu bạn thực hành một chút bạn sẽ nhanh chóng trở nên thành thạo. Hiểu C giúp bạn hiểu cách hoạt động của chương trình và bộ nhớ.
        - Bạn không cần phải đi sâu quá vào cuốn sách (hoặc thậm chí hoàn thành nó). Chỉ cần học cho đến khi bạn cảm thấy thoải mái khi đọc và viết bằng C.
- Python: Hiện đại và rất biểu cảm, tôi học nó vì nó chỉ đơn giản là rất hữu ích và cũng cho phép tôi viết ít code hơn trong một cuộc phỏng vấn.

Đây là các khuyến nghị của tôi. Tất nhiên bạn có thể làm những gì bạn thích.

Có thể bạn không cần, nhưng dưới đây là một số trang web để học một ngôn ngữ mới:

- Exercism
- Codewars
- HackerEarth
- Scaler Topics (Java, C++)

Cho Cuộc Phỏng Vấn Lập Trình của bạn

Bạn có thể sử dụng một ngôn ngữ mà bạn quen thuộc để làm phần lập trình trong cuộc phỏng vấn, nhưng đối với các công ty lớn, những ngôn ngữ sau đây là thích hợp nhất:

- C++
- Java
- Python

Bạn cũng có thể sử dụng các ngôn ngữ sau đây, nhưng hãy tìm hiểu thêm trước. Chúng có thể có bất lợi:

- JavaScript
- Ruby

Dưới đây là một bài viết mà tôi đã viết về việc chọn ngôn ngữ cho cuộc phỏng vấn: Chọn Một Ngôn Ngữ cho Cuộc Phỏng Vấn Lập Trình. Đây là bài viết gốc mà bài đăng của tôi dựa trên: Chọn Một Ngôn Ngữ Lập Trình cho Cuộc Phỏng Vấn

Bạn cần phải quen thuộc với ngôn ngữ và có kiến thức về nó.

Xem thêm về các sự lựa chọn:

- Chọn ngôn ngữ đúng cho cuộc phỏng vấn lập trìnhchoose-the-right-language-for-your-coding-interview/

Xem tài liệu về các ngôn ngữ ở đây

Sách về Cấu Trúc Dữ Liệu và Thuật Toán

Cuốn sách này sẽ tạo nền tảng cho khoa học máy tính của bạn.

Chỉ cần chọn một cuốn, bằng một ngôn ngữ mà bạn quen thuộc. Bạn sẽ đọc và code rất nhiều.

C

- Algorithms in C, Parts 1-5 (Bundle), 3rd Edition
  - Cơ bản, Cấu trúc dữ liệu, Sắp xếp, Tìm kiếm, và Thuật toán đồ thị

Python

- Data Structures and Algorithms in Python
  - của Goodrich, Tamassia, Goldwasser
  - Tôi thích cuốn này. Nó bao gồm mọi thứ cần thiết và hơn thế nữa.
  - Pythonic code (code theo đúng phong cách Python)
  - Cảm nhận của tôi: https://startupnextdoor.com/book-report-data-structures-and-algorithms-in-python/

Java

Lựa chọn ở bạn:

- Goodrich, Tamassia, Goldwasser
  - Data Structures and Algorithms in Java
- Sedgewick và Wayne:
  - Algorithms
  - Khóa học Coursera miễn phí bao gồm cuốn sách (do các tác giả giảng dạy!):
    - Algorithms I
    - Algorithms II

C++

Lựa chọn ở bạn:

- Goodrich, Tamassia, and Mount
  - Data Structures and Algorithms in C++, 2nd Edition
- Sedgewick và Wayne
  - Algorithms in C++, Parts 1-4: Fundamentals, Data Structure, Sorting, Searching
  - Algorithms in C++ Part 5: Graph Algorithms

Sách chuẩn bị phỏng vấn

Bạn không cần phải mua nhiều sách như thế này. Thực sự "Cracking the Coding Interview" có lẽ đã đủ, nhưng tôi đã mua thêm để tạo thêm cơ hội thực hành cho bản thân. Nhưng tôi luôn luôn làm quá mức.

Tôi đã mua cả hai. Chúng đã cho tôi rất nhiều bài tập thực hành.

- ☐ Programming Interviews Exposed: Coding Your Way Through the Interview, 4nd Edition
  - Có câu trả lời bằng C++ và Java
  - Được khuyến khích bởi các khóa hướng dẫn của Google.
  - Đây là một phần luyện tập tốt trước khi bắt đầu với quyển Cracking the Coding Interview
  - Không quá khó, phần lớn các bài toán có lẽ dễ hơn nhiều so với những gì bạn thường thấy trong một buổi phỏng vấn (dựa theo những gì tôi đọc được)
- ☐ Cracking the Coding Interview, 6th Edition
  - Trả lời bằng Java
  - Được khuyến nghị trên Google Careers site
  - Nếu bạn thấy mọi người trích dẫn "The Google Resume", đó là một cuốn sách được thay thế bởi "Cracking the Coding Interview".

Nếu bạn có nhiều thời gian hơn nữa:

Chọn 1:

- Elements of Programming Interviews (C++ version)
- Elements of Programming Interviews in Python
- Elements of Programming Interviews (Java version) - Companion Project - Method Stub and Test Cases for Every Problem in the Book

Đừng phạm sai lầm của tôi

Danh sách này ngày càng dài theo năm tháng và tôi phải thừa nhận là nó ngoài tầm kiểm soát.

Sau đây là 1 vài lỗi tôi đã mắc phải, hy vọng rằng có thể mang lại cho bạn một chút kinh nghiệm.

1. Bạn sẽ không nhớ được tất cả

Tôi đã xem hàng giờ video và viết rất nhiều ghi chú, và chỉ sau vài tháng không còn nhớ chút gì. Tôi đã bỏ ra 3 ngày đọc lại các ghi chú và làm thẻ ghi nhớ để có thể đọc dễ dàng hơn.

Hãy đọc để tránh phạm phải sai lầm tương tự:

Retaining Computer Science Knowledge

2. Sử dụng flash card

Để giải quyết vấn đề, tôi đã viết 1 trang web nhỏ về flash card để thêm các thẻ mới với 2 dạng chính: kiến thức chung và code. Mỗi loại có định dạng riêng.

Tôi đã làm một trang mobile-first (lấy mobile là trọng tâm phát triển trang web) để có thể xem trên điện thoại và máy tính bảng, ở bất cứ đâu.

Tự tạo cho mình hoàn toàn miễn phí:

- Repo của flash card site

TÔI KHÔNG KHUYẾN NGHỊ sử dụng bộ flash card của tôi. Có quá nhiều và hầu hết chúng là những thông tin vặt không cần thiết.

Nhưng nếu bạn không muốn nghe lời tôi, đây là những gì bạn cần:

- Cơ sở dữ liệu flash card của tôi (1200 thẻ):
- Cơ sở dữ liệu flash card của tôi (quá nhiều - 1800 thẻ):

Hãy nhớ rằng tôi đã làm quá mức và các flash card có thể bao gồm cả assembly (ngôn ngữ máy) và Python cho đến cả máy học (machine learning) và thống kê. Như thế là quá nhiều so với những gì cần thiết.

Ghi chú dành cho các flash card: Lần đầu tiên bạn nhận ra bạn biết câu trả lời, đừng đánh dấu là đã biết.Bạn phải xem thẻ tương tự và đưa ra câu trả lời chính xác vài lần trước khi bạn thực sự khẳng định đã nắm được vấn đề. Lặp đi lặp lại việc này sẽ giúp kiến thức được khắc sâu vào não bạn.

Có thể thay thế thẻ ghi nhớ với Anki, đây là ứng dụng mà bạn sẽ thấy tôi khuyến khích sử dụng rất nhiều lần. Nó sử dụng một hệ thống lặp để giúp bạn có thể ghi nhớ được kiến thức.

Đây là ứng dụng cực kì thân thiện với người dùng, có mặt trên tất cả các hệ điều hành, và có hệ thống lưu trữ đồng bộ đám mây. Tốn khoản 25$ cho iOS nhưng miễn phí trên các hệ điều hành khác.

Cơ sở dữ liệu thẻ ghi nhớ của tôi tuân theo chuẩn định dạng của Anki:
https://ankiweb.net/shared/info/25173560 (cảm ơn @xiewenya)

Một số sinh viên đã đề cập đến vấn đề định dạng với khoảng trắng có thể được sửa bằng cách thực hiện như sau: mở bộ thẻ, chỉnh sửa thẻ, nhấp vào thẻ, chọn nút "styling", và thêm "white-space: pre;" vào class thẻ.

Tôi giữ một danh sách xem nhanh các mã của ASCII, OSI stack, định nghĩa về Big-O, và nhiều hơn nữa. Tôi đọc bất cứ khi nào rảnh rỗi.

Khi gặp vấn đề trong lúc code, nghỉ ngơi chừng nửa giờ và đọc lại các thẻ ghi nhớ.

3. Làm Câu Hỏi Phỏng Vấn Lập Trình Trong Khi Bạn Đang Học

ĐIỀU NÀY RẤT QUAN TRỌNG.

Bắt đầu làm câu hỏi phỏng vấn lập trình trong khi bạn đang học cấu trúc dữ liệu và thuật toán.

Bạn cần áp dụng những gì bạn đang học để giải quyết vấn đề, nếu không bạn sẽ quên. Tôi đã mắc phải lỗi này.

Một khi bạn đã học một chủ đề, và cảm thấy khá quen thuộc với nó, ví dụ, **danh sách liên kết**:

1. Mở một trong những Sách chuẩn bị phỏng vấn (hoặc trang web về vấn đề lập trình, được liệt kê dưới đây)
2. Làm 2 hoặc 3 câu hỏi liên quan đến danh sách liên kết.
3. Chuyển sang chủ đề học tiếp theo.

4. Sau đó, quay lại và làm thêm 2 hoặc 3 vấn đề về danh sách liên kết.
5. Làm như vậy với mỗi chủ đề mới bạn học.

Tiếp tục làm các vấn đề trong khi bạn đang học tất cả những điều này, không phải sau này.

Bạn không được tuyển dụng vì kiến thức, mà là cách bạn áp dụng kiến thức.

Có rất nhiều nguồn lực cho điều này, được liệt kê dưới đây. Hãy tiếp tục.

4. Tập trung

Có rất nhiều thứ lấy đi sự tập trung của ta, việc này tốn rất nhiều thời gian. Tập trung và toàn tâm toàn ý rất khó. Bật vài bản nhạc không lời và bạn có thể tập trung hơn.

Những phần không được đề cập

Có vài công nghệ đang thịnh hành nhưng không được đề cập đến, ví dụ:

- Javascript
- HTML, CSS, và các công nghệ thiết kế giao diện người dùng ("front-end").
- SQL

Kế hoạch hàng ngày

Khóa học này bao gồm rất nhiều chủ đề. Mỗi chủ đề có thể mất của bạn vài ngày, hoặc thậm chí hơn một tuần. Điều này phụ thuộc vào lịch trình của bạn.

Mỗi ngày, hãy chọn chủ đề tiếp theo trong danh sách, xem một số video về chủ đề đó, sau đó viết code về cấu trúc dữ liệu hoặc thuật toán đó bằng ngôn ngữ bạn đã chọn cho khóa học này.

Bạn có thể xem code của tôi tại đây:

- C
- C++
- Python

Bạn không cần phải ghi nhớ mọi thuật toán. Bạn chỉ cần hiểu đủ để có thể viết cài đặt của riêng bạn.

Luyện tập các câu hỏi coding

```
Tại sao cái này lại ở đây? Tôi chưa sẵn sàng để phỏng vấn.
```

Then go back and read this.

Lý do bạn cần thực hành giải các bài toán lập trình:

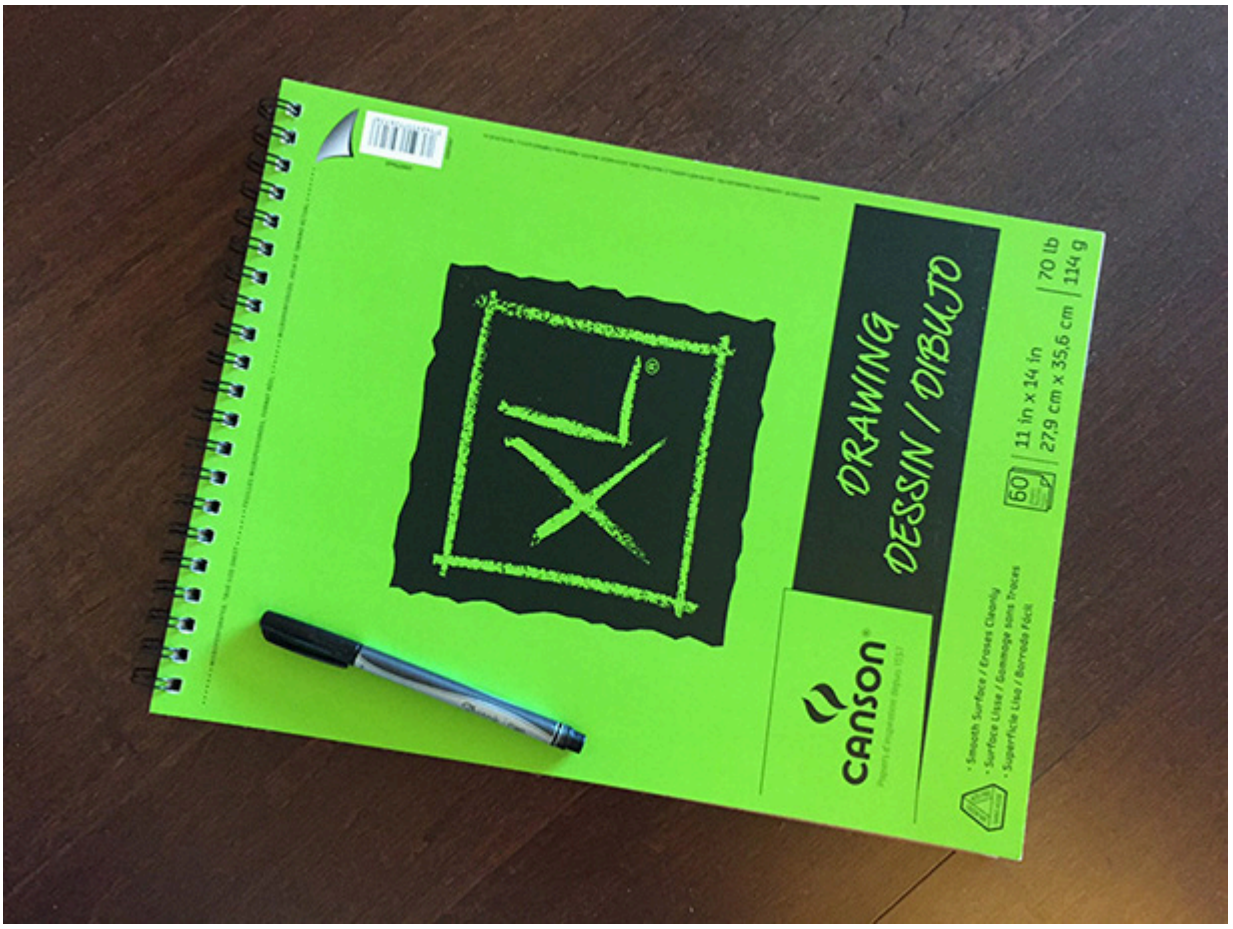
- Nhận diện vấn đề, lựa chọn cấu trúc dữ liệu và thuật toán phù hợp
- Xác định các yêu cầu của vấn đề
- Trình bày cách suy nghĩ của bạn như thể bạn đang trong buổi phỏng vấn
- Lập trình trên bảng trắng hoặc trên giấy, không phải với máy tính

- Đưa ra được tính toán độ phức tạp về thời gian và không gian thực thi của giải pháp của bạn (xem Big-O phía dưới)
- Kiểm tra giải pháp của bạn

Có một lời giới thiệu tuyệt vời về việc giải quyết vấn đề một cách có phương pháp và giao tiếp trong một cuộc phỏng vấn. Bạn cũng sẽ nhận được điều này từ các sách phỏng vấn lập trình, nhưng tôi thấy điều này xuất sắc:: Thuật toán thiết kế canvas

Viết code trên bảng trắng hoặc giấy, không phải máy tính. Kiểm tra với một số đầu vào mẫu. Sau đó gõ nó và kiểm tra nó trên máy tính.

Nếu bạn không có bảng trắng ở nhà, hãy mua một tờ giấy vẽ lớn từ cửa hàng mỹ thuật. Bạn có thể ngồi trên ghế sofa và thực hành. Đây là "bảng trắng sofa" của tôi. Tôi đã thêm bút vào ảnh chỉ để đo lường. Nếu bạn sử dụng bút, bạn sẽ ước gì mình có thể tẩy. Nhanh chóng trở nên lộn xộn. **Tôi sử dụng 1 chiếc bút chì và 1 cục gôm.**



Việc luyện tập giải các bài toán lập trình không phải là việc học thuộc lòng các câu trả lời cho các vấn đề lập trình.

Giải bài toán lập trình

Đừng quên các sách phỏng vấn lập trình quan trọng của bạn ở đây.

Giải quyết vấn đề:

- Cách tìm ra một giải pháp
- Cách phân tích một bài toán Topcoder

Video Câu Hỏi Phỏng Vấn Lập Trình:

- IDeserve (88 videos)
- Tushar Roy (5 playlists)
    - Super for walkthroughs of problem solutions
- Nick White - Giải pháp LeetCode (187 Videos)
    - Giải thích tốt về giải pháp và code
    - Bạn có thể xem nhiều video trong một thời gian ngắn
- FisherCoder - Giải pháp LeetCode

Trang web Thử thách/Thực hành:

- LeetCode
    - Trang web bài toán lập trình yêu thích của tôi. Đáng giá số tiền đăng ký cho 1-2 tháng bạn có thể chuẩn bị.
    - Xem video của Nick White và FisherCoder ở trên để xem hướng dẫn code.
- HackerRank
- TopCoder
- Codeforces
- Codility
- Geeks for Geeks
- AlgoExpert
    - Được tạo bởi các kỹ sư của Google, đây cũng là một nguồn tài nguyên tuyệt vời để trau dồi kỹ năng của bạn.
- Project Euler
    - Rất tập trung vào toán học và không thực sự phù hợp với các cuộc phỏng vấn viết code

Bắt Đầu Nào

Được rồi, nói đủ nhiều rồi, hãy cùng học nào!

Nhưng đừng quên làm các bài toán lập trình ở phía trên trong khi bạn đang học!

Độ phức tạp của thuật toán / Big-O / Phân tích tiệm cận

- Ở đây không có gì phải code, bạn chỉ xem video và ghi chép thôi!
- Có rất nhiều video. Chỉ cần xem đủ cho đến khi bạn hiểu nó. Bạn luôn có thể quay lại và kiểm tra kiến thức.
- Đừng lo lắng nếu bạn không hiểu hết phép toán đằng sau nó.
- Bạn chỉ cần hiểu cách thể hiện độ phức tạp của một thuật toán bằng Big-O.
- ☐ Harvard CS50 - Asymptotic Notation (video)
- ☐ Big O Notations (general quick tutorial) (video)
- ☐ Big O Notation (and Omega and Theta) - best mathematical explanation (video)
- ☐ Skiena (video)
- ☐ UC Berkeley Big O (video)
- ☐ Amortized Analysis (video)
- ☐ TopCoder (includes recurrence relations and master theorem):
    - Computational Complexity: Section 1
    - Computational Complexity: Section 2

- ☐ Cheat sheet
- ☐ [Review] Analyzing Algorithms (playlist) in 18 minutes (video)

Ồ, chắc cũng đủ rồi.

Khi bạn đọc "Cracking the Coding Interview", có một chương về điều này, và cuối chương có một bài kiểm tra để xem nếu bạn có thể xác định độ phức tạp thời gian chạy của các thuật toán khác nhau. Đó là một bài ôn tập và kiểm tra tuyệt vời.

Cấu trúc dữ liệu

- Arrays
  - ☐ Về Arrays:
    - Arrays CS50 Harvard University
    - Arrays (video)
    - UC Berkeley CS61B - Linear and Multi-Dim Arrays - Mảng tuyến tính và mảng đa chiều(video) (Start watching from 15m 32s)
    - Dynamic Arrays - Mảng tùy biến (video)
    - Jagged Arrays - Mảng trong mảng (video)
  - ☐ Cấp phát vector (Mảng có thể thay đổi với khả năng tự điều chỉnh kích cỡ):
    - ☐ Tập sử dụng mảng và con trỏ, dùng phép toán con trỏ để nhảy tới một chỉ mục (index) thay vì sử dụng chỉ mục.
    - ☐ Tạo mảng mới với vùng nhớ được cấp phát sẵn
      - Có thể triển khai mảng số nguyên một cách nhanh chóng, nhưng không sử dụng các tính năng sẵn có
      - Bắt đầu với 16, hoặc số lớn hơn, với cấp số nhân của 2 - 16, 32, 64, 128
    - ☐ size() - Số lượng của các thành phần trong mảng
    - ☐ capacity() - Số lượng tối đa các phần tử mà mảng có thể lưu trữ
    - ☐ is_empty() - Kiểm tra mảng rỗng
    - ☐ at(index) - Trả về phần tử ở vị trí chỉ mục (index), hoặc lỗi nếu rơi ra ngoài chỉ mục
    - ☐ push(item) Thêm vào một phần tử mới
    - ☐ insert(index, item) - Thêm một phần tử mới tại vị trí của chỉ mục, điều chỉnh lại chỉ mục và đưa các phần tử còn lại dịch chuyển theo
    - ☐ prepend(item) - Thêm tại vị trí chỉ mục 0, hay đầu tiên
    - ☐ pop() - trả về phần tử cuối cùng
    - ☐ delete(index) - Xóa phần tử tại chỉ mục, dịch chuyển lại các phần tử trong mảng
    - ☐ remove(item) - Tìm theo giá trị của phần tử và xóa chỉ mục đang lưu trữ cho phần tử này (áp dụng với việc nhiều phần tử có cùng giá trị)
    - ☐ find(item) - Tìm theo giá trị của phần tử và trả về chỉ mục đầu tiên tìm được, -1 nếu không tìm thấy
    - ☐ resize(new_capacity) // private function
      - Khi tới giới hạn của mảng, tăng gấp đôi giá trị độ dài mảng để thay đổi kích thước
      - Khi xóa 1 thành phần, nếu kích thước hiện tại chỉ bằng 1/4 kích thước được cấp phát, thay đổi thành 1/2
  - ☐ Thời gian thực thi

        - O(1) để thêm/xóa tại vị trí cuối (tính luôn cả cấp phát lại để có thêm không gian lưu trữ), đánh chỉ mục, hay cập nhật
        - O(n) để thêm/xóa tại bất cứ đâu
    - ☐ Không gian
        - Liên tục trong bộ nhớ, giúp cải thiện hiệu suất
        - Không gian cần thiết = (Kích cỡ của mảng, thường >= n)* kích thước của 1 phần tử, cho dù là 2n, vẫn xem như O(n)

- Danh sách liên kết (Linked Lists)

    - ☐ Miêu tả:
        - ☐ Linked Lists CS50 Harvard University - this builds the intuition.
        - ☐ Singly Linked Lists (video)
        - ☐ CS 61B - Linked Lists 1 (video)
        - ☐ CS 61B - Linked Lists 2 (video)
        - ☐ [Review] Linked lists in 4 minutes (video)
    - ☐ C Code (video) - Không cần xem toàn bộ video, chỉ phần cấu trúc Node và cấp phát vùng nhớ.
    - ☐ Danh sách liên kết so sánh với Mảng:
        - Core Linked Lists Vs Arrays - Danh sách liên kết Vs Mảng (video)
        - In The Real World Linked Lists Vs Arrays - Trong thực tế, Danh sách liên kết Vs Mảng (video)
    - ☐ Why you should avoid linked lists - Tại sao bạn nên tránh danh sách liên kết(video)
    - ☐ Ghi chú: Bạn cần kiến thức về con trỏ trỏ về con trỏ: (Khi bạn chuyển một con trỏ vào trong 1 thân hàm khiến thay đổi địa chỉ mà con trỏ trỏ về) Trang này giúp bạn có cái nhìn khái quát về con trỏ trỏ tới con trỏ. Tôi không khuyến khích đọc lướt qua danh sách này. Đề tài này rất khó đọc và nắm bắt.
        - Pointers to Pointers - Con trỏ trỏ tới con trỏ - [ ] Cài đặt (Tôi đã thực hiện với con trỏ đuôi và không dùng con trỏ đuôi):
        - ☐ size() - Trả về số lượng các phần tử trong danh sách
        - ☐ empty() - Giá trị luận lý logic, true nếu rỗng
        - ☐ value_at(index) - Trả về phần tử tại vị trí thứ n (danh sách bắt đầu từ 0)
        - ☐ push_front(value) - Thêm phần tử mới vào đầu danh sách
        - ☐ pop_front() - Xóa phần tử đầu tiên và trả về giá trị này
        - ☐ push_back(value) - Thêm phần tử tại cuối danh sách
        - ☐ pop_back() - Xóa phần tử cuối và trả về giá trị
        - ☐ front() - Lấy giá trị của phần tử đầu tiền
        - ☐ back() - Lấy giá trị của phần tử cuối cùng
        - ☐ insert(index, value) - Thêm phần tử mới tại vị trí chỉ mục, phần tử hiện tại sẽ trỏ về phần tử mới tại vị trí chỉ mục này (A->B->C, thêm N tại vị trí B, A->D->B->C, A hiện giờ sẽ trỏ tới D, chỉ mục 1 sẽ trỏ tới D thay vì B)
        - ☐ erase(index) - Xóa node tại vị trí chỉ mục
        - ☐ value_n_from_end(n) - Trả về danh sách từ vị trí thứ n đến cuối danh sách
        - ☐ reverse() - đảo ngược danh sách
        - ☐ remove_value(value) - Xóa dữ liệu đầu tiên được tìm thấy khớp với giá trị được cho
    - ☐ Doubly-linked List
        - Description - Miêu tả danh sách liên kết đôi (video)

        - Không cần phải cài đặt

- Stack
    - ☐ Stacks (video)
    - ☐ [Review] Stacks in 3 minutes (video) - [ ] Sẽ không cài đặt. Cài đặt với mảng là điều hiển nhiên.

- Queue
    - ☐ Queue (video)
    - ☐ Circular buffer/FIFO
    - ☐ [Review] Queues in 3 minutes (video)
    - ☐ Cài đặt sử dụng danh sách liên kết, áp dụng con trỏ đuôi:
        - enqueue(value) - Thêm giá trị ở đuôi
        - dequeue() - Trả về giá trị của dữ liệu được thêm vào xa nhất (thông thường là dữ liệu đầu tiên trong danh sách)
        - empty()
    - ☐ Sử dụng mảng cố định kích thước:
        - enqueue(value) - Thêm giá trị vào cuối mảng
        - dequeue() - Trả về giá trị của dữ liệu được thêm vào xa nhất
        - empty()
        - full()
    - ☐ Chi phí:
        - Không cài đặt đúng việc sử dụng danh sách liên kết khi enqueue tại đầu và dequeue tại đuôi sẽ có chi phí là O(n) bởi vì bạn cần con trỏ next tới giá trị cuối cùng, khiến việc phải chạy qua toàn danh sách mỗi lần dequeue
        - enqueue: O(1) (Không đáng kể, danh sách liên kết và mảng [probing])
        - dequeue: O(1) (danh sách liên kết và mảng)
        - empty: O(1) (danh sách liên kết và mảng)

- Hash table
    - ☐ Video:
        - ☐ Hashing with Chaining (video)
        - ☐ Table Doubling, Karp-Rabin (video)
        - ☐ Open Addressing, Cryptographic Hashing (video)
        - ☐ PyCon 2010: The Mighty Dictionary (video)
        - ☐ PyCon 2017: The Dictionary Even Mightier (video)
        - ☐ (Advanced) Randomization: Universal & Perfect Hashing (video)
        - ☐ (Advanced) Perfect hashing (video)
        - ☐ [Review] Hash tables in 4 minutes (video)
    - ☐ Các khóa học online:
        - ☐ Core Hash Tables - Cơ bản về bảng băm (video)
        - ☐ Data Structures - Cấu trúc dữ liệu (video)
        - ☐ Phone Book Problem - Vấn đề sổ điện thoại (video)

        - [ ] distributed hash tables:
            - Instant Uploads And Storage Optimization In Dropbox - Tải nhanh và tối ưu lưu trữ trong Dropbox (video)
            - Distributed Hash Tables - Phân phối bảng băm(video)

    - [ ] Cài đặt với mảng sử dụng thăm dò tuyến tính:

        - hash(k, m) - m là kích thước của bảng băm
        - add(key, value) - nếu khóa đã tồn tại, cập nhật giá trị
        - exists(key)
        - get(key)
        - remove(key)

Kiến thức ngoài

- Tìm kiếm nhị phân
    - [ ] Tìm kiếm nhị phân (Binary Search) - video
    - [ ] Tìm kiếm nhị phân - Khan Academy (Binary Search) - video
    - [ ] giải thích chi tiết
    - [ ] blueprint
    - [ ] [Review] Binary search in 4 minutes (video)
    - [ ] Cài đặt:
        - Tìm kiếm nhị phân (trên mảng số nguyên đã sắp xếp)
        - Tìm kiếm nhị phân sử dụng đệ quy

- Toán tử trên bit
    - [ ] Bits cheat sheet - bạn nên thuộc lòng nhiều lũy thừa của 2 (từ 2^1 đến 2^16 và 2^32)
    - [ ] Hãy chuẩn bị một nền tảng tốt về các biến đổi bit với các toán tử: &, |, ^, ~, >>, <<
        - [ ] words (thuật ngữ trong kiến trúc máy tính)
        - [ ] Bài mở đầu: Thao tác bit (Bit Manipulation) - video
        - [ ] C Programming Tutorial 2-10: Bitwise Operators (video)
        - [ ] Thao tác bit (Bit Manipulation)
        - [ ] Toán tử trên bit (Bitwise Operation)
        - [ ] Bithacks
        - [ ] The Bit Twiddler
        - [ ] The Bit Twiddler Interactive
        - [ ] Practice Operations
    - [ ] Phần bù 2 và phần bù 1 (2s and 1s complement)
        - Nhị phân: Cộng và trừ (Tại sao chúng ta sử dụng phần bù 2) - video
        - Phần bù 1 (1s Complement)
        - Phần bù 2 (2s Complement)
    - [ ] Đếm tập hợp của các bit
        - 4 cách để đếm các bit trong một byte - video
        - Đếm bit
        - Làm thế nào để đếm số lượng các bit 1 trong một số 32 bit
    - [ ] Đôi giá trị:

        - Swap
    - ☐ Giá trị tuyệt đối:
        - Giá trị tuyệt đối của số nguyên

Cây

- Cây - Ghi chú và kiến thức nền
    - ☐ Chuỗi bài giảng: Cây
    - ☐ Tree Traversal (video)
    - ☐ BFS(breadth-first search) and DFS(depth-first search) (video)
        - BFS - Tìm kiếm ưu tiên chiều rộng (breadth-first search):
            - Các thứ tự duyệt (BFS, sử dụng queue)
            - Độ phức tạp thời gian: O(n)
            - Độ phức tạp không gian: tốt nhất: O(1), tệ nhất: O(n/2)=O(n)
        - DFS - Tìm kiếm ưu tiên chiều sâu (depth-first search)
            - Độ phức tạp thời gian: O(n)
            - Độ phức tạp không gian: tốt nhất: O(log n) - trung bình: bằng chiều cao của cây tệ nhất: O(n)
            - theo thứ tự (inorder) - (DFS: left, self, right)
            - nhánh con trước (postorder) - (DFS: left, right, self)
            - nhánh con sau (preorder) - (DFS: self, left, right)
    - ☐ [Review] Breadth-first search in 4 minutes (video)
    - ☐ [Review] Depth-first search in 4 minutes (video)
    - ☐ [Review] Tree Traversal (playlist) in 11 minutes (video)
- Cây tìm kiếm nhị phân: BST

  BST: Binary search tree - cây tìm kiếm nhị phân.

    - ☐ Ôn lại cây tìm kiếm nhị phân - video
    - ☐ Dẫn nhập - video
    - ☐ MIT (video)
    - C/C++:
        - ☐ Cây tìm kiếm nhị phân - Cài đặt trên C/C++ - video
        - ☐ Cài đặt BST - cấp phát bộ nhớ trên `stack` và `heap` - video
        - ☐ Tìm phần tử nhỏ nhất và lớn nhất trong BST - video
        - ☐ Tìm chiều cao của BST - video
        - ☐ Duyệt cây nhị phân - các chiến lược BFS và DFS - video
        - ☐ Cây nhị phân: duyệt theo cấp - video
        - ☐ Duyệt cây nhị phân: Preorder, Inorder, Postorder - video
        - ☐ Kiểm tra một cậy nhị phân có phải là BST không - video
        - ☐ Xóa một nút trong BST - video
        - ☐ Xác định nút tiếp theo một nút cho trước khi duyệt cây inorder (Inorder Successor in a binary search tree) - video

- [ ] Cài đặt:
  - [ ] `insert` // thêm giá trị vào cây
  - [ ] `get_node_count` // lấy số lượng nút trong cây
  - [ ] `print_values` // In ra giá trị trong cây, từ nhỏ nhất đến lớn nhất
  - [ ] `delete_tree` // Xóa cây
  - [ ] `is_in_tree` // cho biết giá trị cho trước có tồn tại trong cây hay không
  - [ ] `get_height` // cho biết chiều cao của cây(chiều cao của node đơn là 1)
  - [ ] `get_min` // cho biết giá trị nhỏ nhất trong cây
  - [ ] `get_max` // cho biết giá trị lớn nhất trong cây
  - [ ] `is_binary_search_tree` // kiểm tra xem cây cho trước có thỏa mãn điều kiện của một BST không
  - [ ] `delete_value` // xóa một giá trị trong cây
  - [ ] `get_successor` // Trả về phần tử cao nhất trong cây liền sau một giá trị cho trước hoặc -1 nếu không tìm được

- Heap / Priority Queue / Binary Heap
  - hình thức thể hiện như một cây, nhưng chúng thường được lưu trữ theo kiểu tuyến tính (mảng, danh sách liên kết)
  - [ ] Heap
  - [ ] Dẫn nhập - video
  - [ ] Cài đặt căn bản - video
  - [ ] Cây nhị phân - video
  - [ ] Các lưu ý về chiều cao của cây - video
  - [ ] Các toán tử cơ bản - video
  - [ ] Cây nhị phân đầy đủ - video
  - [ ] Mã giả - video
  - [ ] Heap Sort - bắt đầu (video)
  - [ ] Heap Sort (video)
  - [ ] Xây dựng một heap - video
  - [ ] MIT: Heaps and Heap Sort (video)
  - [ ] CS 61B Lecture 24: Priority Queues (video)
  - [ ] Xây dựng Heap với thời gian tuyến tính `O(n)`
  - [ ] [Review] Heap (playlist) in 13 minutes (video)
  - [ ] Tự cài đặt max-heap:
    - [ ] `insert`
    - [ ] `sift_up` - cần thiết cho hàm `insert`.
    - [ ] `get_max` - trả về phần tử lớn nhất mà không xóa nó khỏi heap
    - [ ] `get_size()` - trả về số lượng các phần từ trong một heap
    - [ ] `is_empty()` - trả về `true` nếu heap rỗng
    - [ ] `extract_max` - trả về phần tử lớn nhất và đồng thời xóa nó khỏi heap
    - [ ] `sift_down` - cần thiết cho hàm `extract_max`
    - [ ] `remove(i)` - xóa phần tử tại một vị trí `i` cho trước
    - [ ] `heapify` - tạo một heap từ một mảng các phần tử, cần thiết cho hàm `heap_sort`
    - [ ] `heap_sort()` - nhận vào một mảng chưa sắp xếp, sắp xếp nó tại chỗ (không tốn thêm bộ nhớ) bằng một kỹ thuật sử dụng heap

        - Chú ý: sử dụng min-heap sẽ tiết kiệm được một phép tính (giảm thời gian thực thi thuật toán), nhưng lại tốn gấp đôi bộ nhớ (để chứa kết quả phụ)

Sắp xếp

- ☐ Ghi chú:
    - Cài đặt các thuật toán sắp xếp và với mỗi thuật toán, nắm vững độ phức tạp trong các trường hợp tốt nhất, xấu nhất và trung bình:
        - Bỏ qua sắp xếp nổi bọt (bubble sort) - nó rất tệ - O(n^2), trừ khi n <= 16
    - ☐ Tính ổn định của các thuật toán sắp xếp ("Sắp xếp nhanh (Quicksort) có ổn định không?")
        - Tính ổn định của các thuật toán sắp xếp (wiki)
        - Tính ổn định trong các thuật toán sắp xếp (stackoverflow)
        - Tính ổn định trong các thuật toán sắp xếp (geeksforgeeks)
        - Các thuật toán sắp xếp - Tính ổn định (pdf)
    - ☐ Các thuật toán nào có thể được sử dụng trên danh sách liên kết? mảng? cả hai?
        - Tôi không khuyến khích sắp xếp một danh sách liên kết, nhưng sắp xếp trộn (Merge Sort) có vẻ hữu dụng trong trường hợp đó.
        - Sắp xếp trộn trên danh sách liên kết
- Với sắp xếp vun đống (Heapsort), xem lại cấu trúc Heap ở trên. Sắp xếp vun đống tốt, nhưng không ổn định.
- ☐ Sedgewick - Mergesort (5 videos)
    - ☐ 1. Sắp xếp trộn (Merge Sort)
    - ☐ 2. Bottom-up Mergesort
    - ☐ 3. Sorting Complexity
    - ☐ 4. Comparators
    - ☐ 5. Stability
- ☐ Sedgewick - Quicksort (4 videos)
    - ☐ 1. Quicksort
    - ☐ 2. Selection
    - ☐ 3. Duplicate Keys
    - ☐ 4. System Sorts
- ☐ UC Berkeley:
    - ☐ CS 61B Lecture 29: Sorting I (video)
    - ☐ CS 61B Lecture 30: Sorting II (video)
    - ☐ CS 61B Lecture 32: Sorting III (video)
    - ☐ CS 61B Lecture 33: Sorting V (video)
    - ☐ CS 61B 2014-04-21: Radix Sort(video)
- ☐ Sắp xếp nổi bọt (Bubble Sort) - video
- ☐ Phân tích thuật toán sắp xếp nổi bọt (Analyzing Bubble Sort) - video
- ☐ Sắp xếp chèn và sắp xếp trộn (Insertion Sort, Merge Sort) - video

- [ ] Sắp xếp chèn (Insertion Sort) - video
- [ ] Sắp xếp trộn (Merge Sort) - video
- [ ] Sắp xếp nhanh (Quicksort) - video
- [ ] Sắp xếp chọn (Selection Sort) - video
- [ ] Code cho sắp xếp trộn (Merge sort):
    - [ ] Sử dụng mảng đệm (C)
    - [ ] Sử dụng mảng đệm (Python)
    - [ ] Sắp xếp tại chỗ, không dùng mảng đệm (C++)
- [ ] Cài đặt cho sắp xếp nhanh (Quick sor):
    - [ ] C
    - [ ] Cài đặt khác dùng C
    - [ ] Python
- [ ] [Review] Sorting (playlist) in 18 minutes
    - [ ] Quick sort in 4 minutes (video)
    - [ ] Heap sort in 4 minutes (video)
    - [ ] Merge sort in 3 minutes (video)
    - [ ] Bubble sort in 2 minutes (video)
    - [ ] Selection sort in 3 minutes (video)
    - [ ] Insertion sort in 2 minutes (video)
- [ ] Bài tập cài đặt:
    - [ ] Sắp xếp trộn: O(n log n) trường hợp trung bình và xấu nhất
    - [ ] Sắp xếp nhanh O(n log n) trường hợp trung bình
    - Sắp xếp chọn và sắp xếp chèn đều là O(n^2) cho trường hợp trung bình và xấu nhất.
    - Với sắp xếp vun đống, xem lại cấu trúc Heap ở trên.
- [ ] Không nhất thiết, nhưng tôi khuyến khích xem các phần sau:
    - [ ] Sedgewick - Sắp xếp theo cơ số (Radix Sorts) (6 videos)
        - [ ] 1. Chuỗi trong Java (Strings in Java)
        - [ ] 2. Đếm các khóa đã được lập chỉ mục (Key Indexed Counting)
        - [ ] 3. Sắp xếp cơ số ưu tiên chữ số thấp nhất (Least Significant Digit First String Radix Sort)
        - [ ] 4. Sắp xếp cơ số ưu tiên chữ số cao nhất (Most Significant Digit First String Radix Sort)
        - [ ] 5. 3-way Radix Quicksort: Sắp xếp cơ số kết hợp sắp xếp nhanh, trong đó, chỉ chia mảng ban đầu thành 3 nhóm (thay vì chia theo R nhóm trong sắp xếp cơ số thông thường).
        - [ ] 6. Mảng hậu tố (Suffix Arrays)
    - [ ] Sắp xếp cơ số (Radix Sort)
    - [ ] Sắp xếp cơ số (Radix Sort)(video)

    - [ ] Sắp xếp cơ số, sắp xếp đếm (Counting Sort, thực thi thời gian tuyến tính, nhưng có nhiều giới hạn đối với dự liệu)- video
    - [ ] Sự ngẫu nhiên: Phép nhân ma trận, Sắp xếp nhanh và thuật toán của Freivalds - video
    - [ ] Sắp xếp trong thời gian tuyến tính (Sorting in Linear Time) - video

Tóm lại, dưới đây là biểu đồ trực quan của 15 thuật toán sắp xếp. Nếu bạn cần thêm chi tiết về chủ đề này, xem qua phần "Sắp xếp" trong Đọc thêm về một số đề tài

Đồ thị

Đồ thị có thể được sử dụng để miêu tả nhiều bài toán trong khoa học máy tính, vậy nên phần này cũng khá dài, tương đương với Cây và Sắp xếp.

- Ghi chú:
    - Có 4 cách cơ bản để biểu diễn một đồ thị trong bộ nhớ:
        - Các đối tượng (objects) và con trỏ (pointers)
        - ma trận kề
        - Danh sách kề (adjacency list)
        - bản đồ kề
    - Hãy làm quen với mỗi cách thể hiện và nắm vững ưu, nhược của nó.
    - BFS và DFS - biết về độ phức tạp tính toán của chúng, ước lượng những sự được và mất khi sử dụng, và cài đặt chúng như thế nào
    - Khi được trao cho một bài toán, hãy thử tìm một giải thuật sử dụng đồ thị trước khi nghĩ đến các hướng giải quyết khác.
- [ ] MIT(videos):
    - [ ] Breadth-First Search
    - [ ] Depth-First Search
- [ ] Các bài giảng của giáo sư Skiena, tốt để dẫn nhập:
    - [ ] CSE373 2020 - Lecture 10 - Graph Data Structures (video)
    - [ ] CSE373 2020 - Lecture 11 - Graph Traversal (video)
    - [ ] CSE373 2020 - Lecture 12 - Depth First Search (video)
    - [ ] CSE373 2020 - Lecture 13 - Minimum Spanning Trees (video)
    - [ ] CSE373 2020 - Lecture 14 - Minimum Spanning Trees (con't) (video)
    - [ ] CSE373 2020 - Lecture 15 - Graph Algorithms (con't 2) (video)
- [ ] Đồ thị (ôn tập và mở rộng) (tên video được giữ nguyên vì có quá nhiều thuật ngữ và viết tắt):
    - [ ] 6.006 Single-Source Shortest Paths Problem (video)
    - [ ] 6.006 Dijkstra (video)
    - [ ] 6.006 Bellman-Ford (video)
    - [ ] 6.006 Speeding Up Dijkstra (video)
    - [ ] Aduni: Graph Algorithms I - Topological Sorting, Minimum Spanning Trees, Prim's Algorithm - Lecture 6 (video)
    - [ ] Aduni: Graph Algorithms II - DFS, BFS, Kruskal's Algorithm, Union Find Data Structure - Lecture 7 (video)

    - [ ] Aduni: Graph Algorithms III: Shortest Path - Lecture 8 (video)
    - [ ] Aduni: Graph Alg. IV: Intro to geometric algorithms - Lecture 9 (video)
    - [ ] CS 61B 2014: Weighted graphs (video)
    - [ ] Greedy Algorithms: Minimum Spanning Tree (video)
    - [ ] Strongly Connected Components Kosaraju's Algorithm Graph Algorithm (video)
    - [ ] [Review] Shortest Path Algorithms (playlist) in 16 minutes (video)
    - [ ] [Review] Minimum Spanning Trees (playlist) in 4 minutes (video)

- Khóa học đầy đủ về đồ thị trên Coursera:

    - [ ] Các thuật toán trên đồ thị (video)

- Yegge: Nếu bạn có cơ hội, hãy thử nghiên cứu các thuật toán đẹp hơn:

    - [ ] Thuật toán tìm đường đi ngắn nhất của Dijkstra - xem phần trên - 6.006
    - [ ] A*
        - [ ] Thuật toán tìm kiếm A sao (A*)
        - [ ] Hướng dẫn cài đặt thuật toán tìm đường đi A* - video
        - [ ] Thuật toán tìm đường A* (E01: giải thích thuật toán) - video

- Tôi sẽ viết code cài đặt:

    - [ ] DFS với danh sách kề (đệ quy)
    - [ ] DFS với danh sách kề (lặp với stack)
    - [ ] DFS với ma trận kề (adjacency matrix) (đệ quy)
    - [ ] DFS với ma trận kề (adjacency matrix) (lặp với stack)
    - [ ] BFS với danh sách kề
    - [ ] BFS với ma trận kề
    - [ ] Đường đi ngắn nhất từ một nút đến các nút khác (single-source shortest path) (Dijkstra)
    - [ ] Cây khung (spanning tree) nhỏ nhất
    - Các thuật toán dựa trên DFS (xem lại các video của Aduni):
        - [ ] tìm chu trình (đường đi mà trên đó tồn tại một nút được ghé qua ít nhất 2 lần) trong đồ thị (cần thiết cho sắp xếp topo (topological sort), vì tôi sẽ kiểm tra có chu trình không trước khi bắt đầu sắp xếp)
        - [ ] Sắp xếp topo (topological sort)
        - [ ] Đếm các thành phần liên thông trong một đồ thị
        - [ ] Liệt kê các thành phần liên thông mạnh
        - [ ] Kiểm tra tính song phương của đồ thị (Bipartite graph: dạng đồ thị mà tập định có thể chia thành 2 tập con không chứa phần tử chung, sao cho không có cặp 2 điểm nào trong cùng một tập hợp là kề nhau).

Bạn sẽ biết thêm nhiều ứng dụng của đồ thị trong sách của Skiena (xem danh mục sách bên dưới) và các sách về phỏng vấn.

Kiến thức bổ sung

- Đệ quy

    - [ ] Các bài giảng của đại học Stanford về đệ quy và quay lui:
        - [ ] Lecture 8 | Programming Abstractions (video)

        - [ ] Lecture 9 | Programming Abstractions (video)
        - [ ] Lecture 10 | Programming Abstractions (video)
        - [ ] Lecture 11 | Programming Abstractions (video)
    - Khi nào thích hợp để sử dụng?
    - Đệ quy đuôi tốt hơn hay không như thế nào?
        - [ ] Đệ quy đuôi là gì Tại sao nó không tốt(What Is Tail Recursion Why Is It So Bad)?
        - [ ] Đệ quy đuôi (Tail Recursion) (video)
    - [ ] 5 bước đơn giản để giải quyết bất kỳ vấn đề đệ quy(video)

  Backtracking Blueprint: Java Python

- Quy hoạch động

    - Có thể bạn sẽ không gặp bất kỳ vấn đề lập trình động nào trong cuộc phỏng vấn của mình, nhưng đáng giá để nhận biết một vấn đề có thể là ứng cử viên cho lập trình động.

    - Chủ đề này có thể khá khó khăn, vì mỗi vấn đề có thể giải quyết bằng DP (Dynamic Programming) phải được định rõ là một mối quan hệ đệ quy, và việc đưa ra nó có thể khá khó khăn.

    - Tôi đề nghị xem nhiều ví dụ về các vấn đề DP cho đến khi bạn có sự hiểu biết vững chắc về mô hình liên quan.

    - [ ] Video:

        - [ ] Skiena: CSE373 2020 - Lecture 19 - Introduction to Dynamic Programming (video)
        - [ ] Skiena: CSE373 2020 - Lecture 20 - Edit Distance (video)
        - [ ] Skiena: CSE373 2020 - Lecture 20 - Edit Distance (continued) (video)
        - [ ] Skiena: CSE373 2020 - Lecture 21 - Dynamic Programming (video)
        - [ ] Skiena: CSE373 2020 - Lecture 22 - Dynamic Programming and Review (video)
        - [ ] Simonson: Dynamic Programming 0 (starts at 59:18) (video)
        - [ ] Simonson: Dynamic Programming I - Lecture 11 (video)
        - [ ] Simonson: Dynamic programming II - Lecture 12 (video)
        - [ ] List of individual DP problems (each is short): Dynamic Programming (video)

    - [ ] Ghi chú cho các bài giảng của đại học Yale:

        - [ ] Dynamic Programming

    - [ ] Coursera:

        - [ ] The RNA secondary structure problem (video)
        - [ ] A dynamic programming algorithm (video)
        - [ ] Illustrating the DP algorithm (video)
        - [ ] Running time of the DP algorithm (video)
        - [ ] DP vs. recursive implementation (video)
        - [ ] Global pairwise sequence alignment (video)
        - [ ] Local pairwise sequence alignment (video)

- Lập trình hướng đối tượng

- [ ] Phụ thêm: Seri về UML 2.0 (Optional: UML 2.0 Series) (video)
- [ ] Lập trình hướng đối tượng: Phát triển phần mềm sử dụng UML và Java (21 videos):
  - Có thể bỏ qua phần này nếu bạn đã có kiến thức vững chắc về OOP
  - Lập trình hướng đối tượng: Phát triển phần mềm sử dụng UML và Java
- [ ] Những điểm quan trọng nhất của OOP:
  - [ ] Bob Martin Những điểm quan trọng nhất của lập trình hướng đối tượng và thiết kế linh hoạt (Bob Martin SOLID Principles of Object Oriented and Agile Design) (video)
  - [ ] Thiết kế mẫu trong C# (SOLID Design Patterns in C#) (video)
  - [ ] Những điểm quan trọng nhất (SOLID Principles) (video)
  - [ ] S - Nguyên tắc một trách nhiệm (Single Responsibility Principle)
  - [ ] Mỗi đối tượng chịu một trách nhiệm duy nhất (Single responsibility to each Object)
    - Đọc thêm
  - [ ] O - Nguyên tắc mở đóng (Open/Closed Principal) | Trong thực tế, đối tượng có thể mở rộng chứ không thích hợp cho sửa đổi (On production level Objects are ready for extension for not for modification)
    - Đọc thêm
  - [ ] L - Nguyên tắc thay thế của Liskov (Liskov Substitution Principal) | Nguyên tắc về lớp cơ bản và lớp dẫn xuất (Base Class and Derived class follow 'IS A' principal)
    - Đọc thêm
  - [ ] I - Nguyên tắc phân chia giao diện (Interface segregation principle) | khách hàng không nên triển khai giao diện họ không sử dụng
    - Phân chia giao diện trong 5 phút (Interface Segregation Principle in 5 minutes) (video)
    - Đọc thêm
  - [ ] D -Nguyên tắc phụ thuộc ngược (Dependency Inversion principle) | Giảm bớt phụ thuộc trong tập các đối tượng
    - Tại sao sử dụng nguyên tắc phụ thuộc ngược và tại sao nguyên tắc này quan trọng (Why Is The Dependency Inversion Principle And Why Is It Important)
    - Đọc thêm

- Mẫu thiết kế
  - [ ] Đọc nhanh về UML(Quick UML review) (video)
  - [ ] Học qua các mẫu sau:
    - [ ] strategy
    - [ ] singleton
    - [ ] adapter
    - [ ] prototype
    - [ ] decorator
    - [ ] visitor
    - [ ] factory, abstract factory
    - [ ] facade
    - [ ] observer
    - [ ] proxy
    - [ ] delegate
    - [ ] command
    - [ ] state

        - [ ] memento
        - [ ] iterator
        - [ ] composite
        - [ ] flyweight
    - [ ] Series of videos (27 videos)
    - [ ] Book: Head First Design Patterns
        - Tôi biết có một cuốn sách kinh điển là "Design Patterns: Elements of Reusable Object-Oriented Software" nhưng cuốn Bắt đầu với thiết kế mẫu(Head First Design Patterns) là một lựa chọn tuyệt vời cho người mới bắt đầu với OO.
    - [ ] Ghi nhớ: 101 Thiết kế mẫu và lưu ý dành cho lập trình viên (Handy reference: 101 Design Patterns & Tips for Developers)
    - [ ] Design patterns for humans

- Tổ hợp và Xác Suất

    - [ ] Kỹ năng toán: Làm thế nào để tìm giai thừa, hoán vị và tổ hợp (Math Skills: How to find Factorial, Permutation and Combination) (Choose) (video)
    - [ ] Xác suất (Make School: Probability) (video)
    - [ ] Xác suất và chuỗi Markov (Make School: More Probability and Markov Chains) (video)
    - [ ] Khan Academy:
        - Các khóa học:
            - [ ] Lý thuyết xác suất (Basic Theoretical Probability)
        - 41 video ngắn và đơn giản:
            - [ ] Lý giải về xác suất (Probability Explained) (video)

- NP, NP-Complete và thuật toán xấp xỉ gần đúng

    - Cần biết về các trường hợp nổi tiếng của NP-complete như chuyên du hành doanh nhân hay vấn đề túi đựng đồ nghề (traveling salesman and the knapsack problem) để có thể trả lời khi người phỏng vấn hỏi một cách ẩn ý.
    - Hiểu được NP-complete nghĩa là gì.
    - [ ] Tính toán độ phức tạp (Computational Complexity) (video)
    - [ ] Simonson:
        - [ ] Thuật toán tham lam và giới thiệu NP-complete (Greedy Algs. II & Intro to NP Completeness) (video)
        - [ ] NP Completeness II & Reductions (video)
        - [ ] NP Completeness III (Video)
        - [ ] NP Completeness IV (video)
    - [ ] Skiena:
        - [ ] CSE373 2020 - Lecture 23 - NP-Completeness (video)
        - [ ] CSE373 2020 - Lecture 24 - Satisfiability (video)
        - [ ] CSE373 2020 - Lecture 25 - More NP-Completeness (video)
        - [ ] CSE373 2020 - Lecture 26 - NP-Completeness Challenge (video)
    - [ ] Độ phức tạp: P, NP, NP-completeness, giảm (Complexity: P, NP, NP-completeness, Reductions) (video)
    - [ ] Độ phức tạp: Thuật toán gần đúng (Complexity: Approximation Algorithms) (video)

    - [ ] Độ phức tạp: Thuật toán cố định tham số (Complexity: Fixed-Parameter Algorithms) (video)
    - Peter Norvig đã trình bày thuật toán gần tối ưu cho vấn đề di chuyển của doanh nhân:
        - Jupyter Notebook
    - Trang 1048 - 1140 trong CLRS nếu bạn có nó.

- Cách máy tính xử lý một chương trình

    - [ ] Cách CPU thực thi một chương trình (video)
    - [ ] Cách máy tính tính toán - ALU (video)
    - [ ] Các thanh ghi và RAM (video)
    - [ ] Bộ xử lý trung tâm (CPU) (video)
    - [ ] Hướng dẫn và Chương trình (video)

- Bộ nhớ cache

    - [ ] LRU cache:
        - [ ] Điều kì diệu của LRU cache (100 ngày Google Dev) (The Magic of LRU Cache (100 Days of Google Dev)) (video)
        - [ ] Triển khai LRU(Implementing LRU) (video)
        - [ ] LeetCode - 146 LRU Cache (C++) (video)
    - [ ] CPU cache:
        - [ ] MIT 6.004 L15: Phân cấp bộ nhớ (The Memory Hierarchy) (video)
        - [ ] MIT 6.004 L16: Các vấn đề với bộ nhớ cache (Cache Issues) (video)

- Process và Thread

    - [ ] Khóa học "Khoa học máy tính 162 - Hệ điều hành" (25 video):
        - Về Process và Thread, xem video 1-11
        - Hệ điều hành và lập trình hệ thống (video)
    - Sự khác nhau giữa một Process và một Thread?
    - Học các chủ đề dưới đây:
        - Các vấn đề của Process, Thread và xử lý đồng thời
            - Sự khác nhau giữa Process và Thread
            - Process
            - Thread
            - Locks
            - Mutexes
            - Semaphores
            - Monitors
            - Cách chúng hoạt động
            - Deadlock
            - Livelock
        - Hoạt động của CPU, ngắt, chuyển ngữ cảnh
        - Các kiến trúc xử lý đồng thời được sử dụng cùng với các bộ xử lý đa lõi
        - Paging, segmentation, và virtual memory (video)
        - Interrupts (video)

        - Tài nguyên mà Process cần: (bộ nhớ: các đoạn code, bộ lưu trữ tĩnh, stack, heap cũng như các mô tả của tập tin, nhập/xuất)
        - Tài nguyên mà Thread cần: (chia sẻ các tài nguyên được liệt kê ở trên (trừ stack) với các Thread khác trong cùng Process nhưng mỗi Thread có program counter, stack counter, thanh ghi (registers) và stack của riêng chúng)
        - Forking thực ra là copy on write cho đến khi Process mới ghi vào bộ nhớ, sau đó nó thực hiện hành đồng full copy.
        - Chuyển ngữ cảnh
            - Chuyển ngữ cảnh được bắt đầu như thế nào bởi hệ điều hành và phần cứng bên dưới
    - ☐ Thread trong C++ (danh sách 10 video)
    - ☐ CS 377 Spring '14: Operating Systems from University of Massachusetts
    - ☐ Xử lý đồng thời trong Python (video):
        - ☐ Danh sách ngắn các video về Thread (Short series on threads)
        - ☐ Thread của Python (Python Threads)
        - ☐ Hiểu rõ về Python GIL (Understanding the Python GIL (2010))
            - Tham khảo
        - ☐ David Beazley - Căn bản về xử lý đồng thời trong Python: TRỰC TIẾP! - PyCon 2015 (David Beazley - Python Concurrency From the Ground Up: LIVE! - PyCon 2015)
        - ☐ Các điểm chính trong phần trình bày của David Beazley - Các vấn đề thú vị (Python Asyncio) (Keynote David Beazley - Topics of Interest (Python Asyncio))
        - ☐ Mutex trong Python (Mutex in Python)

- Kiểm thử phần mềm

    - Cần ghi nhớ:
        - Unit test thực hiện như thế nào
        - Các đối tượng giả là gì
        - Integration testing là gì
        - Dependency injection là gì
    - ☐ Kiểm tra phần mềm linh hoạt với Jame Bach (Agile Software Testing with James Bach) (video)
    - ☐ Bài giảng của James Bach về kiểm tra phần mềm (Open Lecture by James Bach on Software Testing) (video)
    - ☐ Steve Freeman - Lập trình hướng tới việc test(Không phải như chúng ta nghĩ) (Test-Driven Development (that's not what we meant)) (video)
        - slides
    - ☐ Dependency injection:
        - ☐ video
        - ☐ Đạo trong kiểm tra (Tao Of Testing)
    - ☐ Viết test thế nào(How to write tests)

- Tìm kiếm và xử lý chuỗi

    - ☐ Sedgewick - Mảng tiền tố (Suffix Arrays) (video)
    - ☐ Sedgewick - Tìm và thay chuỗi (Substring Search) (videos)
        - ☐ 1. Giới thiệu tìm và thay chuỗi (Introduction to Substring Search)

        - [ ] 2. Tìm và thay chuỗi theo Brute-Force (Brute-Force Substring Search)
        - [ ] 3. Knuth-Morris Pratt
        - [ ] 4. Boyer-Moore
        - [ ] 5. Rabin-Karp
    - [ ] Tìm kiếm mẫu trong văn bản (Search pattern in text) (video)

  Nếu bạn cần thêm thông tin chi tiết, hãy đọc qua phần "So khớp chuỗi" trong các mục đọc thêm Đọc thêm về một số đề tài

- Tries

  > Tries: cấu trúc dữ liệu dạng cây cho phép *chèn* và *tìm kiếm* một chuỗi con nhanh (`O(L)`) và một vài lợi thế khác, thích hợp cho một số dạng toán xử lý chuỗi.

    - Chú ý rằng có những loại *trie* khác nhau. Một vài có tiền tố (prefix), một vài loại thì không, và một số thì sử dụng chuỗi thay vì bit để lưu trữ các con đường (từ gốc đến lá).
    - Tôi đã đọc qua code, nhưng sẽ không thực hiện
    - [ ] Sedgewick - Tries (3 videos)
        - [ ] 1. R Way Tries
        - [ ] 2. Ternary Search Tries
        - [ ] 3. Character Based Operations
    - [ ] Notes on Data Structures and Programming Techniques
    - [ ] Các khóa video ngắn:
        - [ ] Introduction To Tries (video)
        - [ ] Performance Of Tries (video)
        - [ ] Implementing A Trie (video)
    - [ ] The Trie: A Neglected Data Structure
    - [ ] TopCoder - Using Tries
    - [ ] Stanford Lecture (real-world use case) (video)
    - [ ] MIT, Advanced Data Structures, Strings (can get pretty obscure about halfway through)

- Cách biểu diễn số thực
    - [ ] Dạng 8-bit đơn giản: Representation of Floating Point Numbers - 1 (video - có một lỗi nhỏ trong phần tính toán, xem thêm ở bên dưới video)

- Unicode
    - [ ] Điều tối thiểu nhất mà mọi lập trình viên bắt buộc phải biết về Unicode và các tập ký tự.
    - [ ] Điều mà mọi lập trình viên nên biết về mã hóa ký tự và tập ký tự để làm việc với văn bản

- Endianness

> Endianness: thứ tự phiên dịch các byte của một chuỗi byte trong bộ nhớ máy tính sang dạng số (4 byte với `int` hoặc 8 byte với `double`). Ví dụ như với 2 byte `0x00` và `0x01` lưu trên bộ nhớ, đọc theo Big-Endian ta được số 1 (0x00001. Đọc theo Little-Endia ta được 256 (0x100). Xem thêm ở các đường link bên dưới.

    - [ ] Big And Little Endian
    - [ ] Big Endian Vs Little Endian (video)
    - [ ] Big And Little Endian Inside/Out (video)
        - Rất nặng về mặt kỹ thuật, dành cho các lập trình viên làm việc với nhân Linux (kernels).
        - Đừng quá bận tâm nếu phần lớn video vượt quá khả năng của bạn. Chỉ cần đoạn nửa đầu video là đủ.

- Mạng máy tính

    - **Nếu bạn có kinh nghiệm với mạng máy tính hoặc muốn trở thành một kỹ sư hệ thống, sẽ có các câu hỏi về chúng**
    - còn nếu không, biết thêm cũng tốt
    - [ ] Khan Academy
    - [ ] UDP and TCP: Comparison of Transport Protocols (video)
    - [ ] TCP/IP and the OSI Model Explained! (video)
    - [ ] Packet Transmission across the Internet. Networking & TCP/IP tutorial. (video)
    - [ ] HTTP (video)
    - [ ] SSL and HTTPS (video)
    - [ ] SSL/TLS (video)
    - [ ] HTTP 2.0 (video)
    - [ ] Video Series (21 videos) (video)
    - [ ] Subnetting Demystified - Part 5 CIDR Notation (video)
    - [ ] Sockets:
        - [ ] Java - Sockets - Introduction (video)
        - [ ] Socket Programming (video)

Tổng kết

```
Phần này sẽ là các video ngắn để bạn ôn tập lại hầu hết các khái niệm quan trọng.
Cũng tốt nếu như bạn muốn bồi dưỡng thường xuyên.
```

- [ ] Các video ngắn 2-3 phút (23 video)
    - Videos
- [ ] Các video ngắn 2-5 phút - Michael Sambol (48 video)
    - Videos
    - Code Examples
- [ ] Sedgewick Videos - Algorithms I
- [ ] Sedgewick Videos - Algorithms II

Lý lịch (Resume) của bạn

- Xem thông tin chuẩn bị CV trong các sách: "Cracking The Coding Interview" và "Programming Interviews Exposed"
- "Đây là những gì một CV TỐT nên có" của Gayle McDowell (tác giả của Cracking the Coding Interview),
    - Lưu ý của tác giả: "Đây là cho một CV tập trung cho nước Mỹ. CV cho Ấn Độ và các quốc gia khác có những kỳ vọng khác nhau, mặc dù nhiều điểm sẽ giống nhau."
- "Hướng dẫn từng bước để tạo CV" của Tech Interview Handbook
    - Hướng dẫn chi tiết về cách thiết lập CV của bạn từ đầu, viết nội dung CV hiệu quả, tối ưu hóa nó, và kiểm tra CV của bạn

Khi bạn tiến gần đến kỳ phỏng vấn

- ☐ Cracking The Coding Interview, Phần 2 (video):
    - Cracking The Code Interview
    - Cracking the Coding Interview - Fullstack Speaker Series

Quy trình phỏng vấn & các bước chuẩn bị tổng quát

- ☐ Cách vượt qua cuộc phỏng vấn kỹ thuật năm 2021
- ☐ Làm rõ việc tuyển dụng kỹ thuật
- ☐ Cách nhận việc tại Big 4:
    - ☐ Cách nhận việc tại Big 4 - Amazon, Facebook, Google & Microsoft (video)
    - ☐ Cách nhận việc tại Big 4.1 (Video tiếp theo)
- ☐ Bộ Cracking The Coding Interview 1:
    - ☐ Gayle L McDowell - Cracking The Coding Interview (video)
    - ☐ Cracking the Coding Interview với tác giả Gayle Laakmann McDowell (video)
- ☐ Cracking the Facebook Coding Interview:
    - ☐ Phương pháp tiếp cận
    - ☐ Hướng dẫn giải quyết vấn đề
- Các khóa học chuẩn bị:
    - Python cho Cấu trúc dữ liệu, Thuật toán, và Phỏng vấn (khóa học trả phí):
        - Một khóa học chuẩn bị phỏng vấn tập trung vào Python, bao gồm cấu trúc dữ liệu, thuật toán, phỏng vấn giả lập, và nhiều hơn nữa.
    - Giới thiệu về Cấu trúc dữ liệu và Thuật toán sử dụng Python (khóa học miễn phí của Udacity):
        - Một khóa học miễn phí tập trung vào Python về cấu trúc dữ liệu và thuật toán.
    - Nanodegree Cấu trúc dữ liệu và Thuật toán! (Nanodegree trả phí của Udacity):
        - Nhận thực hành trực tiếp với hơn 100 bài tập về cấu trúc dữ liệu và thuật toán và hướng dẫn từ một người hướng dẫn chuyên dụng để giúp bạn chuẩn bị cho phỏng vấn và tình huống trong công việc.
    - Hiểu rõ Phỏng vấn Hành vi (khóa học miễn phí của Educative):

        - Nhiều lần, không phải năng lực kỹ thuật của bạn ngăn bạn không nhận được công việc mơ ước, mà là cách bạn thể hiện hành vi trong phỏng vấn.
    - AlgoMonster (khóa học trả phí với nội dung miễn phí):
        - Khóa học tóm tắt cho LeetCode. Bao gồm tất cả các mô hình được tóm gọn từ hàng nghìn câu hỏi.

Phỏng vấn mô phỏng:

- Gainlo.co: Người phỏng vấn mô phỏng từ các công ty lớn - Tôi đã sử dụng dịch vụ này và nó giúp tôi thư giãn cho phần phỏng vấn qua điện thoại và phỏng vấn trực tiếp
- Pramp: Phỏng vấn mô phỏng từ/ với đồng nghiệp - một mô hình ngang hàng để thực hành phỏng vấn
- interviewing.io: Thực hành phỏng vấn mô phỏng với các kỹ sư hàng đầu - phỏng vấn mô phỏng về thiết kế thuật toán/hệ thống với các kỹ sư hàng đầu từ FAANG một cách ẩn danh
- Meetapro: Phỏng vấn mô phỏng với những người phỏng vấn hàng đầu từ FAANG - một nền tảng phỏng vấn mô phỏng/phỏng vấn theo phong cách Airbnb.
- Hello Interview: Phỏng vấn mô phỏng với các huấn luyện viên chuyên nghiệp và AI - phỏng vấn trực tiếp với AI hoặc với các kỹ sư và quản lý từ FAANG.

Hãy nghĩ đến những thứ bạn sẽ được hỏi

Nghĩ sẵn 20 câu hỏi kỹ thuật bạn có thể gặp phải, cùng với danh sách bên dưới. Chuẩn bị 2 đến 3 câu trả lời cho mỗi câu hỏi. Hãy chuẩn bị cả câu chuyện (từ chính kinh nghiệm của bạn), chứ không chỉ một câu trả lời suông.

- Tại sao bạn lại muốn ứng tuyển vị trí này?
- Vấn đề khó chịu nhất bạn từng giải quyết là gì?
- Thử thách lớn nhất bạn từng đối mặt?
- Thiết kế (hệ thống) tốt nhất/tồi nhất bạn từng gặp?
- Những ý tưởng để cải thiện các sản phẩm hiện có.
- Bạn sẽ làm việc như thế nào để đạt hiệu quả tốt nhất, trong trường hợp làm việc một mình và làm theo team?
- Kỹ năng hay kinh nghiệm nào của bạn thích hợp cho công việc này, và tại sao?
- Bạn thích điều gì nhất ở [công việc X/ dự án Y]?
- Thách thức lớn nhất bạn từng gặp phải ở [công việc X/ dự án Y]?
- Lỗi lớn nhất bạn từng gặp phải ở [công việc X/ dự án Y]?
- Bạn học được những gì ở [công việc X/ dự án Y]?
- Những gì bạn cho rằng mình đã có thể làm tốt hơn ở [công việc X/ dự án Y]?

Chuẩn bị câu hỏi dành cho phỏng vấn viên

Một vài câu hỏi của tôi (Tôi có thể đã tìm hiểu trước rồi, nhưng vẫn muốn được nghe ý kiến từ góc nhìn của người phỏng vấn):

- Quy mô của team thế nào?
- Quy trình phát triển của bạn? Waterfall/sprints/agile?
- Áp lực deadline có phổ biến không? Liệu có sự linh hoạt nào không?
- Các quyết định của team được xây dựng như thế nào?
- Khoảng bao nhiêu cuộc họp hàng tuần?

- Bạn có cảm thấy môi trường làm việc của bạn giúp bạn tập trung không?
- Hiện tại bạn đang làm gì?
- Bạn thích gì ở công việc này?
- Cuộc sống làm việc như thế nào?
- Sự cân bằng giữa công việc và cuộc sống của bạn ra sao?

Khi bạn được nhận việc

Chúc mừng!

Hãy tiếp tục rèn luyện.

Bạn không bao giờ thực sự học xong!

```
**************************************************************************************
**************************
**************************************************************************************
**************************

Mọi thứ dưới đây là tùy chọn. Nó không cần thiết cho một cuộc phỏng vấn ở
cấp độ mới vào nghề.
Tuy nhiên, bằng cách nghiên cứu những điều này, bạn sẽ tiếp xúc nhiều hơn
với các khái niệm CS và sẽ được chuẩn bị tốt hơn
cho bất kỳ công việc kỹ sư phần mềm nào. Bạn sẽ trở thành một kỹ sư phần
mềm toàn diện hơn.

**************************************************************************************
**************************
**************************************************************************************
**************************
```

Sách bổ sung

```
Đây là nơi bạn có thể đào sâu vào các chủ đề bạn thấy hứng thú
```

- [ ] The Unix Programming Environment
  - cũ nhưng mà tốt
- [ ] The Linux Command Line: A Complete Introduction
  - một sự lựa chọn hiện đại hơn
- [ ] TCP/IP Illustrated Series
- [ ] Head First Design Patterns

    - dẫn nhập nhẹ nhàng về các mẫu thiết kế.

- ☐ Design Patterns: Elements of Reusable Object-Oriented Software
    - còn được gọi là "Sách của bộ tứ" (Gang Of Four, hoặc GOF)
    - quyển sách cổ điển về mẫu thiết kế.

- ☐ Algorithm Design Manual (Skiena)
    - Như một tài liệu ôn tập và hỗ trợ nhận dạng vấn đề.
    - Danh mục thuật toán thật sự vượt xa độ khó của một cuộc phỏng vấn.
    - Cuốn sách có 2 phần:
        - Giáo trình về cấu trúc dữ liệu và giải thuật:
            - Ưu:
                - Là một bài tổng quát tốt tương đương với các giáo trình khác.
                - Nhiều câu chuyện thú vị từ kinh nghiệm của tác giả trong việc giải quyết các vấn đề thực tế và trong giới học thuật.
                - Code mẫu bằng C.
            - Nhược:
                - Cô đặc và có thể khó hiểu ngang với CLRS, và trong một số chủ đề, CLRS có thể là một tài liệu tốt hơn để tham khảo.
                - Các chương 7, 8, 9 có thể rất vất vả để theo được, vì một vài phần không được giải thích rõ, hoặc là yêu cầu nhiều não hơn những gì tôi có.
                - Đừng hiểu lầm: Tôi thích Skiena, cách dạy học và phong các của ông ấy, nhưng tôi có lẽ không đủ khả năng để tốt nghiệp ở Stony Brook (nơi Skiena giảng dạy).
        - Danh mục thuật toán:
            - Đây là lý do thực sự bạn mua cuốn sách này.
            - Cuốn sách này tốt hơn khi được sử dụng như một tài liệu tham khảo về thuật toán, chứ không phải là một thứ bạn đọc từ đầu đến cuối.
    - Trích dẫn từ Yegge: "Hơn hẳn những quyến sách khác, cuốn này giúp tôi hiểu rõ các bài toán về Graph phổ biến một cách đáng kinh ngạc và quan trọng như thế nào - chúng nên là một phần trong các công cụ của bất kỳ lập trình viên nào. Quyển sách đồng thời cũng bao phủ các cấu trúc dữ liệu cơ bản, các thuật toán sắp xếp. Đó là một điểm cộng. Nhưng phần quý giá thật sự nằm ở nửa sau, chính là bách khoa toàn thư ngắn gọn về hàng triệu bài toán hữu dụng và vô số cách để giải quyết chúng, trình bày sơ lược. Mỗi trang đều có một hình minh họa, giúp người đọc dễ ghi nhớ hơn. Đó là một cách tốt để định dạng và phân loại các bài toán".
    - Có thể thuê quyển sách này trên Kindle
    - Câu trả lời cho các bài tập trong sách:
        - Solutions
    - Errata

- Algorithm (Jeff Erickson)

- ☐ Write Great Code: Volume 1: Understanding the Machine
    - Quyển này được xuất bản năm 2004, phần nào đã lỗi thời, nhưng nó vẫn là một tài liệu tuyệt vời để tìm hiểu về máy tính một cách ngắn gọn. - Tác giả phát minh ra HLA (High Level

Assembly), vậy nên hãy hãy chú ý một chút về các ví dụ và định nghĩa trong sách. Tuy không được sử dụng rộng rãi, nhưng đó là một ví dụ hiện đại về hợp ngữ (assembly).
    - Những chương này rất đáng đọc để xây dựng cho bạn một nền tảng tốt (giữ nguyên gốc tiếng Anh):
        - Chapter 2 - Numeric Representation
        - Chapter 3 - Binary Arithmetic and Bit Operations
        - Chapter 4 - Floating-Point Representation
        - Chapter 5 - Character Representation
        - Chapter 6 - Memory Organization and Access
        - Chapter 7 - Composite Data Types and Memory Objects
        - Chapter 9 - CPU Architecture
        - Chapter 10 - Instruction Set Architecture
        - Chapter 11 - Memory Architecture and Organization

- ☐ Introduction to Algorithms - **Chú ý:** Đọc cuốn này chỉ có một ít giá trị. Đây là một tổng hợp xuất sắc về giải thuật và cấu trúc dữ liệu, nhưng nó không dạy cho bạn cách viết code xuất sắc. Để làm một lập trình viên giỏi, bạn đồng thời phải có khả năng phát triển một giải pháp một cách hiệu quả nữa.

    - Trích lời Yegge: "Nhưng nếu bạn muốn đến với buổi phỏng vấn một cách có chuẩn bị, vậy hãy hoãn đơn xin ứng tuyển lại cho đến khi bạn hoàn tất quyển sách này"
    - Half.com là một trang hữu dụng để tìm sách với giá tốt.
    - Đôi được gọi là CLR, hoặc là CLRS (trích chữ cái đầu trong tên của các tác giả), vì Stein (một trong 4 tác giả, S trong CLRS) nhập cuộc trễ

- Kiến trúc Máy tính, Ấn bản thứ Sáu: Một cách tiếp cận định lượng

    - Đối với một cách tiếp cận phong phú hơn, cập nhật hơn (2017), nhưng dài hơn

Thiết kế hệ thống, Khả năng mở rộng, Xử lý dữ liệu

- **Bạn có thể sẽ bị hỏi câu hỏi liên quan đến thiết kế hệ thống nếu có hơn 4 năm kinh nghiệm**

- Khả năng mở rộng và Thiết kế hệ thống là các chủ đề rất rộng, với nhiều vấn đề và tài liệu liên quan, bởi vì có rất nhiều vấn đề cần phải giải quyết khi thiết kế các hệ thống phần mềm (hoặc phần cứng) có thể mở rộng được. Cần phải đầu tư một chút thời gian cho vấn đề này.

- Lời khuyên từ Yegge:

    - Khả năng mở rộng
        - Trích xuất từ cơ sở dữ liệu lớn về một giá trị độc nhất
        - Chuyển đổi từ một tập dữ liệu sang tập khác
        - Xử lý một khối lượng dữ liệu đồ sộ
    - Thiết kế hệ thống
        - Tập hợp các tính năng
        - Giao diện
        - Lớp phân cấp
        - Thiết kế hệ thống dưới những ràng buộc
        - Đơn giản và vững chắc

        - Đánh đổi (từ gốc tradeoff, chấp nhận bỏ một vài tính năng nhỏ để có được những lợi ích khác như tốc độ xử lý...)
        - Phân tích và tối ưu hiệu suất sử dụng

- ☐ **BẮT ĐẦU TỪ ĐÂY**: The System Design Primer
- ☐ System Design from HiredInTech - Thiết kế hệ thống bởi HiredInTech
- ☐ Làm thế nào để trả lời các câu hỏi liên quan tới thiết kế trong phỏng vấn kỹ thuật? (How Do I Prepare To Answer Design Questions In A Technical Inverview)
- ☐ 8 điều bạn nên biết trước khi bắt đầu một buổi phỏng vấn về Thiết kế hệ thống (8 Things You Need to Know Before a System Design Interview)
- ☐ Chuẩn hóa trong cơ sở dữ liệu (Database Normalization - 1NF, 2NF, 3NF and 4NF) (video)
- ☐ Phỏng vấn về thiết kế hệ thống (System Design Interview) - Có rất nhiều tài liệu trong link nay. Hãy đọc qua các bài viết và các ví dụ. Tôi có liệt kê sau đây:
- ☐ Làm thế nào để dẫn đầu trong cuộc phỏng vấn về thiết kế hệ thống (How to ace a systems design interview)
- ☐ Những con số ai cũng nên biết (Numbers Everyone Should Know)
- ☐ Mất bao lâu để làm một chuyển đổi ngữ cảnh? (How long does it take to make a context switch)
- ☐ Các luồng xử lý trong trung tâm dữ liệu (Transactions Across Datacenters)(video)
- ☐ Một hướng dẫn đơn giản về lý thuyết CAP (A plain English introduction to CAP Theorem)
- ☐ MIT 6.824: Distributed Systems, Spring 2020 (20 videos)
- ☐ Thuật toán đồng thuận:
    - ☐ Paxos - Paxos Agreement - Computerphile (video)
    - ☐ Raft - An Introduction to the Raft Distributed Consensus Algorithm (video)
        - ☐ Easy-to-read paper
        - ☐ Infographic
- ☐ Consistent Hashing
- ☐ NoSQL Patterns
- ☐ Khả năng mở rộng:
    - Bạn không cần tất cả chúng. Chỉ cần chọn 1 vài cái bạn có hứng thú
    - ☐ Tầm nhìn chung (Great overview)(video)
    - ☐ Các seri ngắn:
        - Nhân bản (Clones)
        - Cơ sở dữ liệu (Database)
        - Cache
        - Bất đồng bộ (Asynchronism)

- [ ] Kiến trúc web và hệ thống phân tán có khả năng mở rộng (Scalable Web Architecture and Distributed Systems)
- [ ] Fallacies of Distributed Computing Explained
- [ ] Jeff Dean - Xây dựng hệ thống phần mềm tại Google và các bài học rút ra được (Jeff Dean - Building Software Systems At Google and Lessons Learned)(video)
- [ ] Giới thiệu về kiến trúc hệ thống có thể mở rộng (Introduction to Architecting Systems for Scale)
- [ ] Mở rộng game trên di động nhắm tới khách hàng trên toàn thế giới sử dụng App Engine và Cloud Datastore (Scaling mobile games to a global audience using App Engine and Cloud Datastore)(video)
- [ ] How Google Does Planet-Scale Engineering for Planet-Scale Infra (video)
- [ ] Sự quan trọng của thuật toán (The Importance of Algorithms)
- [ ] Chia nhỏ cơ sở dữ liệu thành từng phần nhỏ và nhanh hơn, dễ quản lý hơn (Sharding)
- [ ] Khả năng mở rộng của Facebook (Scale at Facebook) - (2009)
- [ ] Khả năng mở rộng của Facebook (2012), "Xây dựng cho cả tỷ người dùng" (Scale at Facebook (2012), "Building for a Billion Users" -)(video)
- [ ] Lập trình cho cả khả năng phát triển trong tương lai (Engineering for the Long Game) - Astrid Atkinson Keynote - (video)
- [ ] 7 năm mở rộng của Youtube trong 30 phút (7 Years Of YouTube Scalability Lessons In 30 Minutes)
    - video
- [ ] Paypal đã mở rộng thế nào để đáp ứng hơn tỷ lượt giao dịch mỗi ngày với 8VMs (How PayPal Scaled To Billions Of Transactions Daily Using Just 8VMs)
- [ ] Làm thế nào để xóa lặp trong dữ liệu (How to Remove Duplicates in Large Datasets)
- [ ] A look inside Etsy's scale and engineering culture with Jon Cowie) (video)
- [ ] Cùng nhìn nhận cách Etsy mở rộng và phong cách lập trình với Jon Cowie (What Led Amazon to its Own Microservices Architecture - Điều gì đưa Amazon tới kiến trúc microservices
- [ ] Nén hay không nén, đây là câu hỏi dành cho Uber (To Compress Or Not To Compress, That Was Uber's Question)
- [ ] Trong trường hợp nào truy vấn phỏng đoán được thực thi? (When Should Approximate Query Processing Be Used?)
- [ ] Google's Transition From Single Datacenter, To Failover, To A Native Multihomed Architecture
- [ ] Kỹ thuật tối ưu hình ảnh để phục vụ hàng triệu yêu cầu mỗi ngày (The Image Optimization Technology That Serves Millions Of Requests Per Day)
- [ ] Trình bày ngắn về kiến trúc Patreon (A Patreon Architecture Short)
- [ ] Tinder: Làm thế nào mà một trong những hệ thống hẹn hò lớn nhất quyết định bạn sẽ nhìn thấy ai tiếp theo (Tinder: How Does One Of The Largest Recommendation Engines Decide Who You'll See Next?)
- [ ] Design Of A Modern Cache
- [ ] Facebook đã mở rộng thế nào để đáp ứng việc trình diễn video trực tiếp (Live Video Streaming At Facebook Scale)
- [ ] Hướng dẫn cơ bản cho việc mở rộng đến hơn 11 triệu người dùng với Amazon AWS (A Beginner's Guide To Scaling To 11 Million+ Users On Amazon's AWS)

    - [ ] Sử dụng docker ảnh hưởng tới độ trễ như thế nào? (How Does The Use Of Docker Effect Latency?)
    - [ ] Có thể xem AMP như 1 đối thủ với Google không? (Does AMP Counter An Existential Threat To Google?)
    - [ ] Một cái nhìn 360 độ về toàn bộ Netflix Stack (A 360 Degree View Of The Entire Netflix Stack)
    - [ ] Độ trễ ảnh hưởng tới doanh thu của bạn - Làm sao để khắc phục? (Latency Is Everywhere And It Costs You Sales - How To Crush It)
    - [ ] Serverless (rất dài, chỉ nên dùng file gist)
    - [ ] Điều gì làm nên sức mạnh của Instagram: Hàng trăm phần tử, hàng tá các công nghệ (What Powers Instagram: Hundreds of Instances, Dozens of Technologies)
    - [ ] Kiến trúc của Salesforce - Làm thế nào để xử lý 1.3 tỷ giao dịch mỗi ngày (Salesforce Architecture - How They Handle 1.3 Billion Transactions A Day)
    - [ ] Kiến trúc của ESPN khi mở rộng - Xử lý 100000 thông tin mỗi giây (ESPN's Architecture At Scale - Operating At 100,000 Duh Nuh Nuhs Per Second)
    - [ ] Xem qua "Messaging, Serialization, and Queueing Systems" phía dưới để hiểu các công nghệ có thể kết nối các dịch vụ cùng lúc thế nào.
    - [ ] Twitter:
        - O'Reilly MySQL CE 2011: Jeremy Cole, "Big and Small Data at @Twitter" (video)
        - Timelines at Scale
    - Để có thêm thông tin, xem seri "Mining Massive Datasets" trong seri video.

- [ ] Thực hành thêm về thiết kế hệ thống: Sau đây là vài ý tưởng có thể thực hiện trên giấy, mỗi ý tưởng đều có tư liệu đi cùng để hiểu rõ nó thực thi thế nào trong thế giới thực.

    - review: The System Design Primer
    - Thiết kế hệ thống bởi HiredInTech (System Design from HiredInTech)
    - cheat sheet
    - Các bước thực hiện:
        1. Hiểu vấn đề và phạm vi của vấn đề:
            - Định nghĩa các trường hợp sử dụng cụ thể với sự trợ giúp từ người phỏng vấn
            - Đề xuất thêm tính năng
            - Bỏ đi những phần mà người phỏng vấn cho là không còn nằm trong phạm vi yêu cầu
            - Giả sử khả năng khả dụng cao, thêm vào như một tính năng
        2. Suy nghĩ về các ràng buộc:
            - Hỏi xem có bao nhiêu yêu cầu mỗi tháng
            - Hỏi xem có bao nhiêu yêu cầu mỗi giây (hoặc họ đưa nó cho bạn hoặc bạn phải tự tính toán ra)
            - So sánh tần số đọc và ghi
            - Sử dụng luật 80/20 khi ước lượng
            - Bao nhiêu dữ liệu được viết mỗi giây
            - Toàn bộ kho lưu trữ dữ liệu cần cho 5 năm
            - Bao nhiêu dữ liệu được đọc mỗi giây
        3. Thiết kế trừu tượng:
            - Lớp (dịch vụ, dữ liệu, caching)
            - Cấu trúc hệ thống: tải cân bằng, chuyển thông điệp

        - Cái nhìn tổng quan về thuật toán chủ chốt để chạy các dịch vụ
        - Đưa ra hướng giải quyết cho hiện tượng nghẽn cổ chai
    - Bài tập:
        - Thiết kê một hệ thống cung cấp ID ngẫu nhiên (Design a random unique ID generation system)
        - Thiết kế một cơ sở dữ liệu khóa-giá trị (Design a key-value database)
        - Thiết kế một hệ thống chia sẻ ảnh (Design a picture sharing system)
        - Thiết kế một hệ thống hổ trợ ra quyết định (Design a recommendation system)
        - Thiết kế một hệ thống làm ngắn URL (Design a URL-shortener system: copied from above)
        - Thiết kế một hệ thống cache (Design a cache system)

Học thêm

> Tiêu đề của các video, các thuật ngữ cao cấp xin được giữ nguyên. Một số thuật ngữ có thể dịch được, nhưng người dịch không đủ vốn từ đề diễn đạt chúng một cách ngắn gọn như trong tiếng Anh, nên cũng xin phép cho qua.

```
Tôi đã thêm chúng để giúp bạn trở thành một kỹ sư phần mềm toàn diện và để
bạn biết về một số
công nghệ và thuật toán, vì vậy bạn sẽ có một bộ công cụ lớn hơn.
```

- Trình biên dịch
    - How a Compiler Works in ~1 minute (video)
    - Harvard CS50 - Compilers (video)
    - C++ (video)
    - Understanding Compiler Optimization (C++) (video)
- Emacs và vi(m)
    - Phải nắm vững một trình chỉnh sửa văn bản thông dụng trên Unix.
    - vi(m):
        - Editing With Vim 01 - Installation, Setup, and The Modes (video)
        - VIM Adventures
        - 4 video:
            - The vi/vim editor - Lesson 1
            - The vi/vim editor - Lesson 2
            - The vi/vim editor - Lesson 3
            - The vi/vim editor - Lesson 4
        - Using Vi Instead of Emacs
    - emacs:
        - Basics Emacs Tutorial (video)
        - 3 video:
            - Emacs Tutorial (Beginners) -Part 1- File commands, cut/copy/paste, cursor commands

        - Emacs Tutorial (Beginners) -Part 2- Buffer management, search, M-x grep and rgrep modes
        - Emacs Tutorial (Beginners) -Part 3- Expressions, Statements, ~/.emacs file, and packages
      - Evil Mode: Or, How I Learned to Stop Worrying and Love Emacs (video)
      - Writing C Programs With Emacs
    - The Absolute Beginner's Guide to Emacs (video by David Wilson)
    - The Absolute Beginner's Guide to Emacs (notes by David Wilson)

- Các công cụ chạy trên dòng lệnh của Unix
    - bash
    - cat
    - grep
    - sed
    - awk
    - curl or wget
    - sort
    - tr
    - uniq
    - strace
    - tcpdump

- Lý thuyết thông tin
    - Khan Academy
    - ☐ Hiểu thêm về chuỗi Markov:
        - Core Markov Text Generation
        - Core Implementing Markov Text Generation
        - Project = Markov Text Generation Walk Through
    - Xem thêm ở chuỗi bài từ đại học MIT 6.050J Information và Entropy bên dưới.

- Parity & Hamming Code (videos)
    - Intro
    - Parity
    - Hamming Code:
        - Error detection
        - Error correction
    - Error Checking

- Mã hóa
    - Xem các video bên dưới.
    - Hãy chắc chắn đã xem qua các video đề lý thuyết thông tin trước.
    - Khan Academy Series
    - Cryptography: Hash Functions
    - Cryptography: Encryption

- Thuật toán nén
  - Hãy chắc chắn đã xem qua các video đề lý thuyết thông tin trước.
  - ☐ Computerphile (videos):
    - Compression
    - Entropy in Compression
    - Upside Down Trees (Huffman Trees)
    - EXTRA BITS/TRITS - Huffman Trees
    - Elegant Compression in Text (The LZ 77 Method)
    - Text Compression Meets Probabilities
  - Compressor Head videos
  - (optional) Google Developers Live: GZIP is not enough!
- Bảo mật
  - MIT (23 videos)
    - Introduction, Threat Models
    - Control Hijacking Attacks
    - Buffer Overflow Exploits and Defenses
    - Privilege Separation
    - Capabilities
    - Sandboxing Native Code
    - Web Security Model
    - Securing Web Applications
    - Symbolic Execution
    - Network Security
    - Network Protocols
    - Side-Channel Attacks
- Trình dọn rác

> Trình dọn rác (garbage collection) là một tính năng của các ngôn ngữ lập trình cấp cao, trong đó hệ thông tự động thu hồi vùng nhớ của các data (biến, đối tượng) không còn được sử dụng nữa, và cấp phát chúng cho các data mới. Trước khi có tính năng này, lập trình viên phải quản lý vùng nhớ thủ công, tự xin cấp phát và tự giải phóng.

  - GC in Python (video)
  - Deep Dive Java: Garbage Collection is Good!
  - Deep Dive Python: Garbage Collection in CPython (video)
- Lập trình song song
  - Coursera (Scala)
  - Efficient Python for High-Performance Parallel Computing (video)
- Messaging, Serialization, và Queueing Systems
  - Thrift
    - Tutorial

    - Protocol Buffers
        - Tutorials
    - gRPC
        - gRPC 101 for Java Developers (video)
    - Redis
        - Tutorial
    - Amazon SQS (queue)
    - Amazon SNS (pub-sub)
    - RabbitMQ
        - Get Started
    - Celery
        - First Steps With Celery
    - ZeroMQ
        - Intro - Read The Manual
    - ActiveMQ
    - Kafka
    - MessagePack
    - Avro

- A*
    - A Search Algorithm
    - A* Pathfinding (E01: algorithm explanation) (video)

- Fast Fourier Transform
    - An Interactive Guide To The Fourier Transform
    - What is a Fourier transform? What is it used for?
    - What is the Fourier Transform? (video)
    - Divide & Conquer: FFT (video)
    - Understanding The FFT

- Bloom Filter
    - Cho một bộ lọc Bloom với m bit và k hàm băm, cả việc chèn và kiểm tra thành viên đều là O(k)
    - Bloom Filters (video)
    - Bloom Filters | Mining of Massive Datasets | Stanford University (video)
    - Tutorial
    - How To Write A Bloom Filter App

- HyperLogLog
    - How To Count A Billion Distinct Objects Using Only 1.5KB Of Memory

- Locality-Sensitive Hashing
    - Được sử dụng để kiểm tra tính tương đồng của các tài liệu.

    - Trái ngược lại với MD5 và SHA, chúng vốn được dùng để xác định chắc chắn rằng 2 tài liệu hoàn toàn trùng khớp với nhau.
    - Simhashing (hopefully) made simple

- van Emde Boas Trees

    - Divide & Conquer: van Emde Boas Trees (video)
    - MIT Lecture Notes

- Augmented Data Structures

    - CS 61B Lecture 39: Augmenting Data Structures

- Balanced search trees

    - Biết ít nhất một dạng cây nhị phân tìm kiếm cân bằng (và biết cách cài đặt nó)

    - "Trong các loại cây nhị phân cân bằng, cây AVL và 2/3 đã "ra đi", và cây đỏ-đen (red-black) có vẻ khá phổ biến. Một dạng cây tự quản khá thú vị là *splay tree*, sử dụng phép xoay nút để chuyển bất kỳ nút nào lên trên gốc" - Skiena

    - Do đó, tôi quyết định cài đặt splay tree. Từ những gì tôi đọc được, bạn không cần phải cài đặt một cây nhị phân trong cuộc phỏng vấn. Nhưng tôi muốn chắc chắn về kiến thức của mình, và cũng phải đối diện với sự thật rằng splay tree khá khó.

        - splay tree: insert, search, delete functions
        - Nếu bạn muốn cài đặt cây đỏ-đen thì có thể thử sức các phép toán này: search and insertion functions, skipping delete

    - Tôi muốn học nhiều hơn về B-Tree, bởi vì nó được sử dụng rộng rãi với các tập dữ liệu rất lớn.

    - Self-balancing binary search tree

    - ☐ **AVL trees**

        - Trong thực tế: Từ những gì mà tôi đọc được, chúng không được sử dụng nhiều, nhưng tôi có thể nghĩ đến vài ứng dụng: cây AVL hỗ trợ O(log n) cho tìm kiếm, chèn và xóa nút. Nó được cân bằng một cách chặt chẽ hơn so với cây đỏ-đen, dẫn đến việc chèn và xóa chậm hơn, nhưng tìm kiếm sẽ nhanh hơn. Điều đó khiến nó thích hợp với các bộ dữ liệu được xây dựng một lần và tái sử dụng nhiều lần, ví dụ như từ diển trong ngôn ngữ thông thường (hoặc các bộ từ vựng trong lập trình, biên dịch)
        - MIT AVL Trees / AVL Sort (video)
        - AVL Trees (video)
        - AVL Tree Implementation (video)
        - Split And Merge
        - [Review] AVL Trees (playlist) in 19 minutes (video)

    - ☐ **Splay trees**

        - Trong thực tế: Splay tree được sử dụng để cài đặt cache, bộ cấp phát bộ nhớ, định tuyến, bộ dọn rác, trình nén dữ liệu, ropes (thay thế hoặc đại diện cho một chuỗi lớn),

trong Windows NT (bộ nhớ ảo, mạng và hệ thống file.)
    - CS 61B: Splay Trees (video)
    - MIT Lecture: Splay Trees:
        - Gets very mathy, but watch the last 10 minutes for sure.
        - Video

- ☐ **Red/black trees**
    - Đây là một biến thể của cây 2/3 tree (xem bên dưới)
    - Trong thực tế: Cây Đỏ-đen đảm bảo tốc độ thực thi ok trong trường hợp xấu nhất cho các phép toán: chèn, xóa, tìm kiếm. Điều đó không chỉ khiến nó có giá trị trong các ứng dụng thời gian thực, mà còn là phần cơ bản để xây dựng nhiều dạng cấu trúc dữ liệu nâng cao, ví dụ như, trong tính toán địa lý, nhiều cấu trúc dữ liệu được xây dựng dựa trên cây đỏ-đen, thuật toán Lập lịch tuyệt đối công bằng (Completely Fair Scheduler) sử dụng trong nhân Linux cũng dùng cây đỏ-đen. Trên Java 8, HashMap cũng được điều chỉnh lại, thay vi dùng LinkedList để chứa các phần tử giống nhau (trùng hashcode), người ta dùng một cây đỏ-đen.
    - Aduni - Algorithms - Lecture 4 (link jumps to the starting point) (video)
    - Aduni - Algorithms - Lecture 5 (video)
    - Red-Black Tree
    - An Introduction To Binary Search And Red Black Tree
    - [Review] Red-Black Trees (playlist) in 30 minutes (video)
- ☐ **2-3 search trees**
    - Trong thực tế: cây 2-3 chèn nhanh hơn, nhưng tìm kiếm chậm hơn (chiều cao của cây cao hơn so với AVL)
    - Bạn sẽ hiếm khi sử dụng cây 2-3, vì cài đặt của chúng sử dụng các dạng nút khác nhau. Vì vậy, người ta dùng cây đỏ-đen nhiều hơn.
    - 23-Tree Intuition and Definition (video)
    - Binary View of 23-Tree
    - 2-3 Trees (student recitation) (video)
- ☐ **2-3-4 Trees (aka 2-4 trees)**
    - Trong thực tế: Với mỗi cây 2-4, có nhiều cây đỏ-đen tương ứng với cùng thứ tự nút. Phép chèn và xóa trên cây 2-4 cũng tương đương với phép đổi màu và xoay trên cây đỏ-đen. Nó khiến cho cây 2-4 trở thành một công cụ hiệu quả để hiểu được logic phía sau cây đỏ-đen, và đó là tại sao nhiều sách dẫn nhập thuật toán lại giới thiệu cây 2-4 trước cây đỏ-đen. **cây 2-4 không thường được dùng trong thực tế**
    - CS 61B Lecture 26: Balanced Search Trees (video)
    - Bottom Up 234-Trees (video)
    - Top Down 234-Trees (video)
- ☐ **N-ary (K-ary, M-ary) trees**
    - chú ý: N hay K chính là yếu tố cho phân nhánh (số nhánh tối đa)
    - Cây nhị phân là 2-ary, trong đó, số nhánh tối đa (branching factor) là 2
    - Cây 2-3 là 3-ary

        - K-Ary Tree

    - ☐ **B-Trees**

        - fun fact: điều đó vẫn còn là một bí ẩn, nhưng B ở đây có thể là Boeing, Balanced (được cân bằng) hoặc Bayer (người đồng sáng tạo)
        - Trong thực tế: B-tree được sử dụng rộng rãi trong các cơ sở dữ liệu. Hầu hết các hệ thống file sử dụng B-tree (hoặc biến thể của nó). Thêm vào đó, loại cấu trúc cho phép truy cập ngẫu nhiên nhanh chóng vào một file bất kỳ. Vấn đề còn lại là điều chỉnh địa chỉ khối vùng nhớ của file thành địa chỉ vật lý của ổ đĩa (hoặc địa chỉ *cylinder-head-sector*). (or perhaps to a cylinder-head-sector) address.
        - B-Tree
        - B-Tree Datastructure
        - Introduction to B-Trees (video)
        - B-Tree Definition and Insertion (video)
        - B-Tree Deletion (video)
        - MIT 6.851 - Memory Hierarchy Models (video) - giải thích cache mau quên (cache-oblivious) B-Trees, một cấu trúc dữ liệu rất thú vị. - 37 phút đầu tiên rất nặng kỹ thuật, có thể bỏ qua
        - [Review] B-Trees (playlist) in 26 minutes (video)

- k-D Trees

    - Tốt để tìm số điểm nằm trong một hình chữ nhật hoặc một đối tượng đa chiều.
    - Rất thích hợp cho thuật toán tìm láng giềng gần nhất (k-nearest neighbors, một thuật toán trong khai thác dữ liệu)
    - kNN K-d tree algorithm (video)

- Skip lists

    - "Đó là một dạng cấu trúc dữ liệu được sùng bái" - Skiena
    - Randomization: Skip Lists (video)
    - For animations and a little more detail

- Network Flows

    - Ford-Fulkerson in 5 minutes — Step by step example (video)
    - Ford-Fulkerson Algorithm (video)
    - Network Flows (video)

- Disjoint Sets & Union Find

    - UCB 61B - Disjoint Sets; Sorting & selection (video)
    - Sedgewick Algorithms - Union-Find (6 videos)

- Math for Fast Processing

    - Integer Arithmetic, Karatsuba Multiplication (video)
    - The Chinese Remainder Theorem (used in cryptography) (video)

- Treap
  - Sự kết hợp giữa cây nhị phân và heap.
  - Treap
  - Data Structures: Treaps explained (video)
  - Applications in set operations
- Linear Programming (videos)
  - Linear Programming
  - Finding minimum cost
  - Finding maximum value
  - Solve Linear Equations with Python - Simplex Algorithm
- Geometry, Convex hull (videos)
  - Graph Alg. IV: Intro to geometric algorithms - Lecture 9
  - Geometric Algorithms: Graham & Jarvis - Lecture 10
  - Divide & Conquer: Convex Hull, Median Finding
- Discrete math
  - Computer Science 70, 001 - Spring 2015 - Discrete Mathematics and Probability Theory
  - Discrete Mathematics by Shai Simonson (19 videos)
  - Discrete Mathematics By IIT Ropar NPTEL

- Machine Learning
  - ☐ Tại sao nên học ML?
    - ☐ How Google Is Remaking Itself As A Machine Learning First Company
    - ☐ Large-Scale Deep Learning for Intelligent Computer Systems (video)
    - ☐ Deep Learning and Understandability versus Software Engineering and Verification by Peter Norvig
  - ☐ Google's Cloud Machine learning tools (video)
  - ☐ Google Developers' Machine Learning Recipes (Scikit Learn & Tensorflow) (video)
  - ☐ Tensorflow (video)
  - ☐ Tensorflow Tutorials
  - ☐ Practical Guide to implementing Neural Networks in Python (using Theano)
  - Các khóa học:
    - Great starter course: Machine Learning - videos only - see videos 12-18 for a review of linear algebra (14 and 15 are duplicates)
    - Neural Networks for Machine Learning
    - Google's Deep Learning Nanodegree
    - Google/Kaggle Machine Learning Engineer Nanodegree
    - Self-Driving Car Engineer Nanodegree
  - Tài nguyên:
    - Sách:
      - Python Machine Learning

        - Data Science from Scratch: First Principles with Python
        - Introduction to Machine Learning with Python
    - Machine Learning for Software Engineers
    - Data School: http://www.dataschool.io/

--

Đọc thêm về một số đề tài

Tôi thêm những phần này để củng cố các kiến thức đã được trình bày ở trên, nhưng không muốn đưa chúng vào danh sách trên, vì đã quá nhiều rồi. Cũng có hơi vượt mức cần thiết. Nhưng dù sao, bạn muốn trúng tuyển mà phải không?

- **SOLID**
    - ☐ Bob Martin SOLID Principles of Object Oriented and Agile Design (video)
    - ☐ S - Single Responsibility Principle | Single responsibility to each Object
        - more flavor
    - ☐ O - Open/Closed Principle | On production level Objects are ready for extension but not for modification
        - more flavor
    - ☐ L - Liskov Substitution Principle | Base Class and Derived class follow 'IS A' Principle
        - more flavor
    - ☐ I - Interface segregation principle | Clients should not be forced to implement interfaces they don't use
        - Interface Segregation Principle in 5 minutes (video)
        - more flavor
    - ☐ D -Dependency Inversion principle | Reduce the dependency In composition of objects.
        - Why Is The Dependency Inversion Principle And Why Is It Important
        - more flavor
- **Union-Find**
    - Overview
    - Naive Implementation
    - Trees
    - Union By Rank
    - Path Compression
    - Analysis Options
- **Đi sâu hơn vào quy hoạch động** (videos)
    - 6.006: Dynamic Programming I: Fibonacci, Shortest Paths
    - 6.006: Dynamic Programming II: Text Justification, Blackjack
    - 6.006: DP III: Parenthesization, Edit Distance, Knapsack
    - 6.006: DP IV: Guitar Fingering, Tetris, Super Mario Bros.
    - 6.046: Dynamic Programming & Advanced DP
    - 6.046: Dynamic Programming: All-Pairs Shortest Paths
    - 6.046: Dynamic Programming (student recitation)

- **Xử lý đồ thị nâng cao** (videos)
    - Synchronous Distributed Algorithms: Symmetry-Breaking. Shortest-Paths Spanning Trees
    - Asynchronous Distributed Algorithms: Shortest-Paths Spanning Trees
- MIT **Xác suất** (nặng toán học, và hãy đi chậm chậm, sẽ tốt cho các vấn đề toán học khác) (videos):
    - MIT 6.042J - Probability Introduction
    - MIT 6.042J - Conditional Probability
    - MIT 6.042J - Independence
    - MIT 6.042J - Random Variables
    - MIT 6.042J - Expectation I
    - MIT 6.042J - Expectation II
    - MIT 6.042J - Large Deviations
    - MIT 6.042J - Random Walks
- Simonson: Approximation Algorithms (video)
- **So khớp chuỗi**
    - Rabin-Karp (videos):
        - Rabin Karps Algorithm
        - Precomputing
        - Optimization: Implementation and Analysis
        - Table Doubling, Karp-Rabin
        - Rolling Hashes, Amortized Analysis
    - Knuth-Morris-Pratt (KMP):
        - TThe Knuth-Morris-Pratt (KMP) String Matching Algorithm
    - Boyer–Moore string search algorithm
        - Boyer-Moore String Search Algorithm
        - Advanced String Searching Boyer-Moore-Horspool Algorithms (video)
    - Coursera: Algorithms on Strings
        - Khởi đầu khá tốt. Nhưng theo thời gian, khóa học đi qua thuật toán KMP, và trở nên phức tạp hơn mức cần thiết.
        - Trình bày về trie rất hay.
        - Có thể bỏ qua.
- **Sắp xếp**
    - Bài giảng của Stanford về sắp xếp:
        - Lecture 15 | Programming Abstractions (video)
        - Lecture 16 | Programming Abstractions (video)
    - Shai Simonson:
        - Algorithms - Sorting - Lecture 2 (video)
        - Algorithms - Sorting II - Lecture 3 (video)
    - Steven Skiena lectures on sorting:
        - CSE373 2020 - Mergesort/Quicksort (video)
        - CSE373 2020 - Linear Sorting (video)
- NAND To Tetris: Build a Modern Computer from First Principles

Các chuỗi Video

Hãy ngồi xuống và thưởng thức.

- List of individual Dynamic Programming problems (each is short)
- x86 Architecture, Assembly, Applications (11 videos)
- MIT 18.06 Linear Algebra, Spring 2005 (35 videos)
- Excellent - MIT Calculus Revisited: Single Variable Calculus
- Skiena lectures from Algorithm Design Manual - CSE373 2020 - Analysis of Algorithms (26 videos)
- UC Berkeley 61B (Spring 2014): Data Structures (25 videos)
- UC Berkeley 61B (Fall 2006): Data Structures (39 videos)
- UC Berkeley 61C: Machine Structures (26 videos)
- OOSE: Software Dev Using UML and Java (21 videos)
- MIT 6.004: Computation Structures (49 videos)
- Carnegie Mellon - Computer Architecture Lectures (39 videos)
- MIT 6.006: Intro to Algorithms (47 videos)
- MIT 6.033: Computer System Engineering (22 videos)
- MIT 6.034 Artificial Intelligence, Fall 2010 (30 videos)
- MIT 6.042J: Mathematics for Computer Science, Fall 2010 (25 videos)
- MIT 6.046: Design and Analysis of Algorithms (34 videos)
- MIT 6.824: Distributed Systems, Spring 2020 (20 videos)
- MIT 6.851: Advanced Data Structures (22 videos)
- MIT 6.854: Advanced Algorithms, Spring 2016 (24 videos)
- Harvard COMPSCI 224: Advanced Algorithms (25 videos)
- MIT 6.858 Computer Systems Security, Fall 2014
- Stanford: Programming Paradigms (27 videos)
- Introduction to Cryptography by Christof Paar
    - Course Website along with Slides and Problem Sets
- Mining Massive Datasets - Stanford University (94 videos)
- Graph Theory by Sarada Herke (67 videos)

Các khóa học khoa học máy tính

- Directory of Online CS Courses
- Directory of CS Courses (many with online lectures)

Algorithms implementation

- Multiple Algorithms implementation by Princeton University

Các bài nghiên cứu

  - Có nhiều bài nghiên cứu của Google và các bài nghiên cứu rất nổi tiếng.
  - Đọc hết các bài nghiên cứu tốn rất nhiều thời gian, tôi khuyên bạn nên chọn lọc theo đề tài cụ thể.

- Love classic papers?
- 1978: Process giao tiếp tuần tự (1978: Communicating Sequential Processes)
  - Triển khai với Go (implemented in Go)
- 2003: Hệ thống tập tin của Google (2003: The Google File System)
  - thay thế bởi Colossus năm 2012
- 2004: MapReduce: Đơn giản hóa xử lý dữ liệu trên phân vùng lớn (Simplified Data Processing on Large Clusters)
  - gần như bị thay thế bởi Cloud Dataflow?
- 2006: Bigtable: Một Hệ thống Lưu trữ Phân tán cho Dữ liệu Cấu trúc
- 2006: Dịch vụ Khóa Chubby cho Hệ thống Phân tán Kết nối Lỏng lẻo
- 2007: Dynamo: Key-value store Có khả năng Sẵn sàng cao của Amazon
  - Bài báo Dynamo đã khởi đầu cho cuộc cách mạng NoSQL
- 2007: Những gì mà lập trình viên nên biết về bộ nhớ (rất dài, ngay cả tác giả cũng khuyến khích bỏ qua vài phần) (What Every Programmer Should Know About Memory (very long, and the author encourages skipping of some sections))
  - 2012: AddressSanitizer: Kiểm tra nhanh tính đúng đắn của địa chỉ (A Fast Address Sanity Checker):
    - Bài nghiên cứu
    - video
  - 2013: Spanner: Google's Globally-Distributed Database:
    - Bài nghiên cứu
    - video
- 2014: Máy học: Lợi ích và những vấn đề cần tránh (Machine Learning: The High-Interest Credit Card of Technical Debt)

- 2015: Continuous Pipelines at Google
- 2015: Sẵn sàng cho thay đổi lớn: Xây dựng hạ tầng dữ liệu của Google dành cho quảng cáo (High-Availability at Massive Scale: Building Google’s Data Infrastructure for Ads)
- 2015: Tensorflow: Áp dụng máy học với dữ liệu lớn trên hệ thống phân tán tính toán song song (Large-Scale Machine Learning on Heterogeneous Distributed Systems)
- 2015: Lập trình viên tìm code như thế nào: Một trường hợp điển hình (How Developers Search for Code: A Case Study)
- 2016: Borg, Omega, and Kubernetes
- More papers: 1,000 papers